

Quality Assurance and Testing

This page documents how we tested the Power Apps prototype against the acceptance criteria from Assignment 1. The aim of this testing was not only to check whether each button works, but also to verify that the main civilian and responder workflows behave correctly, that data is written to SharePoint as expected, and that known Power Apps limitations are clearly documented.

Our testing combines Power Apps Test Studio, manual evidence, SharePoint backend checks, Power Apps Monitor observations, and accessibility review. This is because some parts of our app use modern controls and external platform features that are not always fully supported by Test Studio.

Numbering guide

- AC refers to an acceptance criterion from Assignment 1.
- TS refers to a Test Suite created in Power Apps Test Studio.
- TC refers to an individual Test Case inside a Test Studio suite.
- BE refers to Backend Evidence from SharePoint or Power Apps data source screenshots.
- PE refers to Performance Evidence collected from Power Apps Monitor.
- AE refers to Accessibility Evidence from the app interface.
- IF refers to an Issue/Fix record found during implementation or testing.

Testing Strategy

Our testing strategy was based on the main user workflows in the prototype. We focused first on the highest-risk and highest-value functions: submitting an emergency request, validating required input, responder login, incident queue filtering, dispatch actions, support requests, false report handling, and completion of an incident.

Power Apps Test Studio was used where the interaction could be reliably automated, such as button selection, screen navigation, and visibility checks. For areas where Test Studio had limitations, especially modern dropdown controls, location permission behaviour, and SharePoint backend verification, we used manual evidence and screenshots. This approach follows the practical testing advice from the course: use automated tests where possible, but document limitations and use reasonable workarounds when the tool cannot fully automate a scenario.

Each test was mapped back to the Assignment 1 acceptance criteria so that the final implementation could be checked against the original requirements rather than only against the

current interface.

Acceptance Criteria Traceability Matrix

This matrix is our formal test plan. It maps each acceptance criterion from Assignment 1 to one or more test cases or evidence items. The final result column records whether the implemented prototype satisfies that criterion.

AC ID	Acceptance Criterion	Related Test / Evidence	Result	Comment
AC1	Civilian can submit an emergency request through the app.	TC1, BE2	Pass	Request can be submitted and stored in SharePoint.
AC2	User's current location is captured and included in the request.	TC1, BE2, IF1	Pass with limitation	Location is captured as latitude and longitude due to platform restrictions on reverse geocoding.
AC3	The app prevents incomplete emergency submissions.	TC2	Pass	Confirm action is blocked or user remains on the confirmation screen if required information is missing.
AC4	Responder can create or use a responder account.	TC3, TC4, BE4	Pass	Account details are stored in ResponderDetails.
AC5	Responder can view active incidents.	TC6	Pass	Active queue shows New and On the Way incidents only.

AC6	Responder can accept/dispatch a new incident.	TC7, BE3	Pass	Status updates to On the Way and tracking screen opens.
AC7	Responder can request additional support.	TC8, BE3	Pass	Support flag is updated and warning appears on tracking screen.
AC8	Responder can flag a false report using verification information.	TC10, BE3	Pass	False report is a manual responder judgement, not automatic system decision.
AC9	Responder can mark an incident as done.	TC11, BE3	Pass	Done incidents are removed from the active queue.
AC10	The prototype connects Power Apps screens with SharePoint backend data.	BE1, BE2, BE3, BE4	Pass	Both EmergencyIncidents and ResponderDetails are connected and used.

Power Apps Test Studio Test Suites and Results

Test Summary Table

TestID	Test case	Acceptance Criteria Covered	Method	Result	Evidence
TC1	Submit emergency request successfully.	AC1, AC2	Test Studio + manual screen evidence	Pass	Fig-TC1, BE2

TC2	Prevent submission when required information is missing.	AC3	Test Studio	Pass	Fig-TC2
TC3	Create responder account successfully.	AC4	Test Studio + backend evidence	Pass	Fig-TC3, BE4
TC4	Login with valid responder credentials.	AC5	Test Studio	Pass	Fig-TC4
TC5	Reject invalid responder login.	AC6	Test Studio	Pass	Fig-TC5
TC6	Show active incidents in correct priority order.	AC7	Test Studio + manual evidence	Pass	Fig-TC6
TC7	Accept new incident and move to tracking.	AC8	Test Studio + backend evidence	Pass	Fig-TC7, BE3
TC8	Request additional support and show warning.	AC9	Test Studio + manual evidence	Pass	Fig-TC8, BE3
TC9	Open On the Way incident directly in tracking screen.	AC10	Test Studio	Pass	Fig-TC9
TC10	Flag incident as false report.	AC11	Test Studio + manual evidence	Pass	Fig-TC10, BE3
TC11	Complete dispatch and remove incident from active queue.	AC12	Test Studio + backend evidence	Pass	Fig-TC11, BE3

Test Evidence

TC1 — Submit emergency request successfully

Purpose:

This test verifies that a civilian can enter the emergency request flow, view their captured location, select an incident type, and submit an emergency request successfully.

Expected result:

The app should navigate from the Civilian role to the confirmation screen, display the current GPS coordinates, allow the user to select an incident type, and then show the Emergency Status / Safety Guidance screen after the request is confirmed.

Actual result:

Pass. Test Studio confirmed the stable automated part of the flow, including role selection, navigation to the confirmation screen, and location display. The final incident type selection and Confirm action were verified manually because Test Studio did not reliably record the modern dropdown control.

Evidence:

Figure TC1.1. Automated Test Studio result showing that the civilian request flow reaches the confirmation screen and displays the current location.

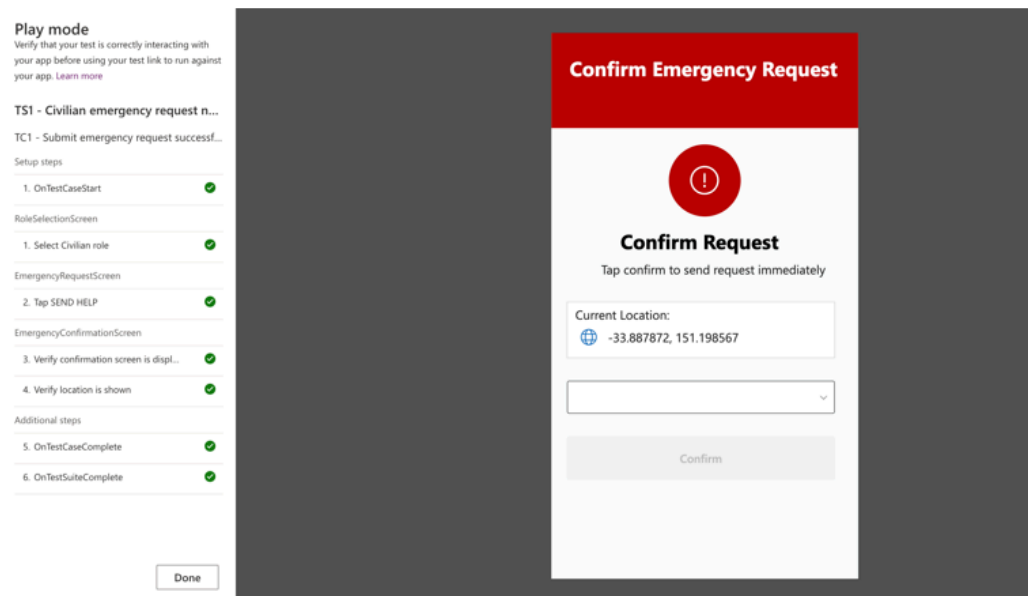


Figure TC1.2. Manual evidence showing that an incident type was selected and the Confirm button became available.

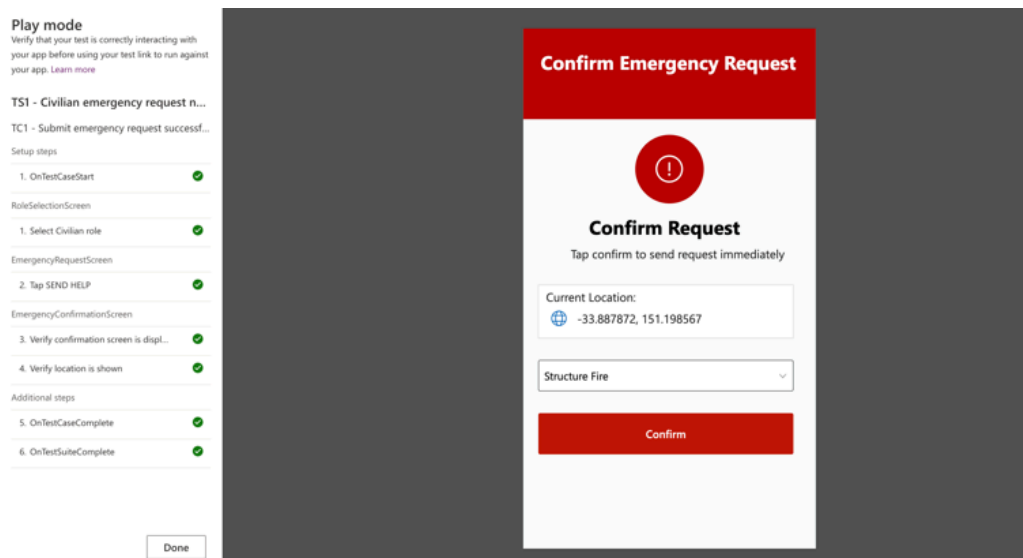
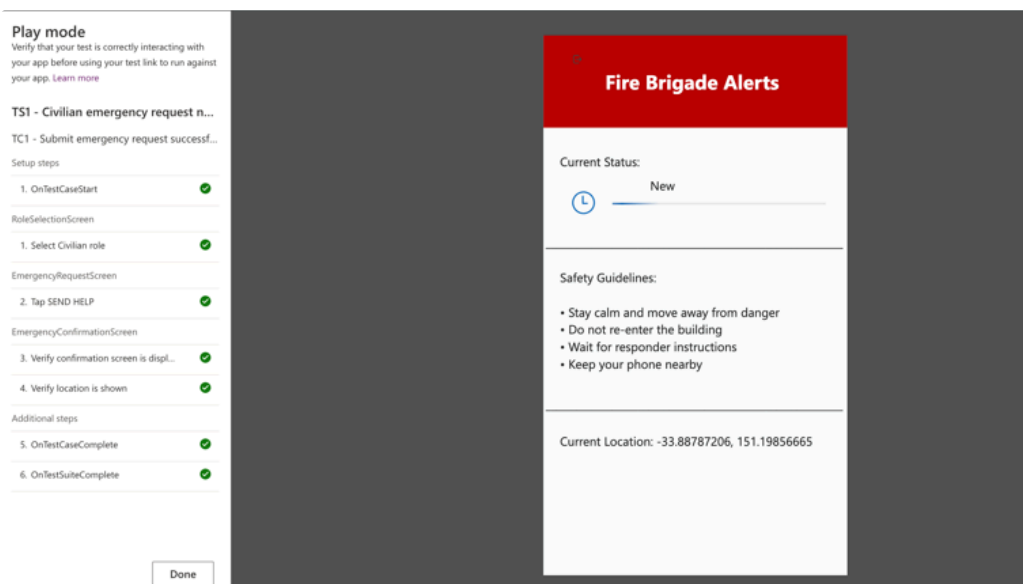


Figure TC1.3.Manual evidence showing the Emergency Status / Safety Guidance screen after the request was submitted.



TC2 — Prevent submission when required information is missing(edge/error-handling test)

Purpose:

This test verifies that the app prevents a civilian from submitting an incomplete emergency request when the incident type has not been selected.

Expected result:

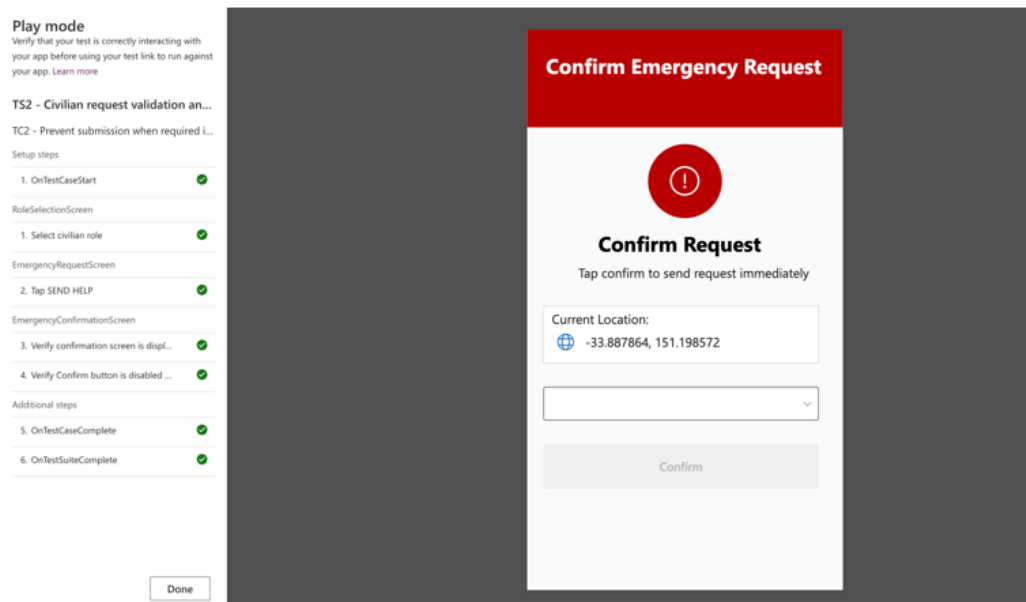
After the user reaches the confirmation screen, the incident type field should remain blank and the Confirm button should stay disabled. The user should not be able to submit the request until a valid incident type is selected.

Actual result:

Pass. Test Studio confirmed that the civilian could reach the confirmation screen, and the Confirm button remained disabled when no incident type was selected.

Evidence:

Figure TC2.Test Studio result showing that the Confirm button remains disabled when no incident type is selected.



TC3 — Create responder account successfully

Purpose:

This test verifies that a responder can open the sign-up page, enter the required account details, and create a responder account successfully.

Expected result:

The app should allow the responder to access the sign-up page from the responder flow. After valid details are entered, the account should be created and the app should show a successful account creation message.

Actual result:

Pass with documented workaround. Test Studio successfully confirmed that the responder sign-up page could be opened from the responder flow. However, Test Studio could not reliably enter values into the sign-up text input fields, so the form entry and final account creation were verified manually. The responder account was created successfully and a matching record was stored in the ResponderDetails SharePoint list.

Evidence:

Figure TC3.1. Automated Test Studio result showing that the responder sign-up page opens successfully from the responder flow.

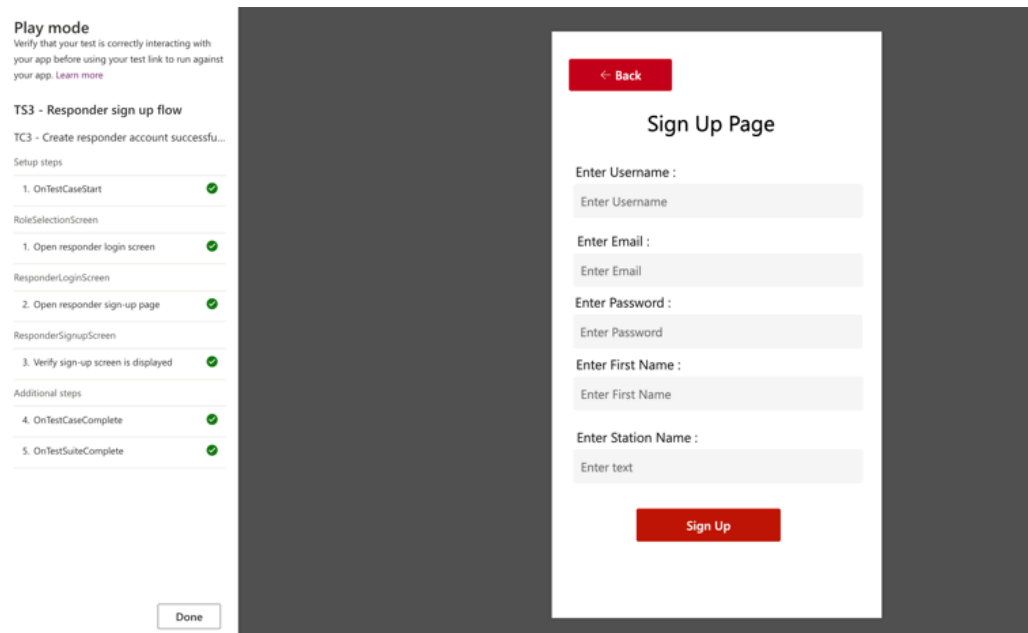


Figure TC3.2. Manual evidence showing the responder sign-up form filled with test account details.

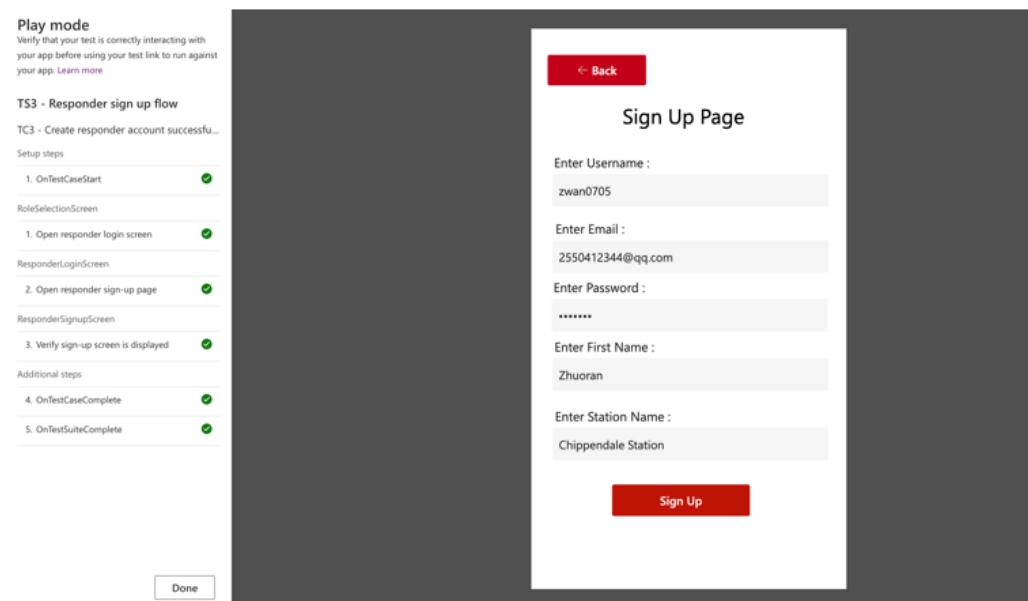
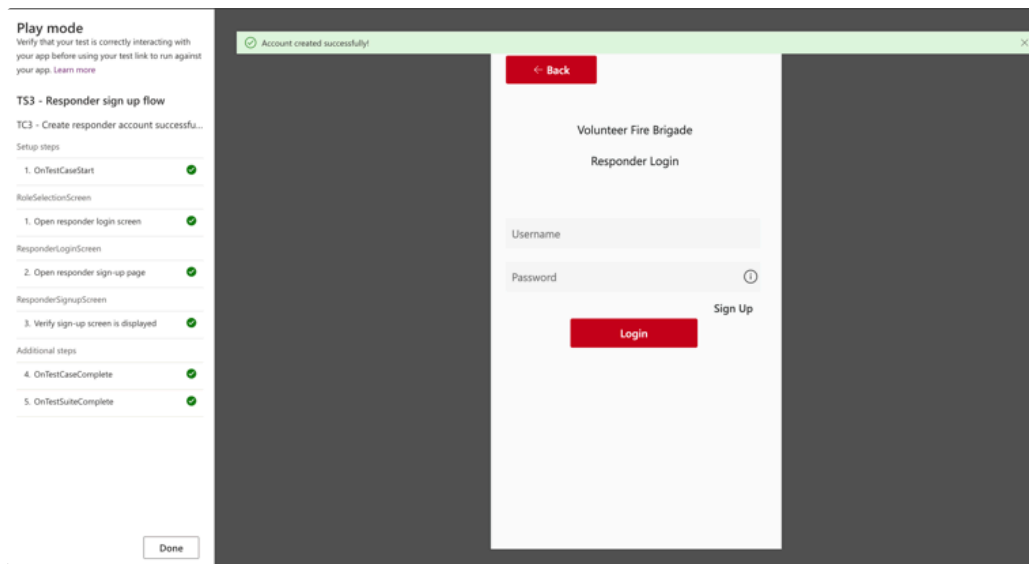


Figure TC3.3. Manual evidence showing the successful account creation message after submitting the sign-up form.



TC4 — Login with valid responder credentials

Purpose:

This test verifies that an existing responder can open the responder login screen, enter valid credentials, and continue into the responder workflow.

Expected result:

The app should open the responder login screen from the role selection page. After a valid username and password are entered, the app should accept the login and allow the responder to continue to the incident workflow.

Actual result:

Pass with documented workaround. Test Studio confirmed that the responder login screen opened correctly from the role selection page. However, Test Studio did not reliably enter values into the login input fields, so the credential entry and successful login were verified manually. Manual testing confirmed that the valid test responder account could log in and access the incident workflow.

Evidence:

Figure TC4.1. Automated Test Studio result showing that the responder login screen opens correctly.

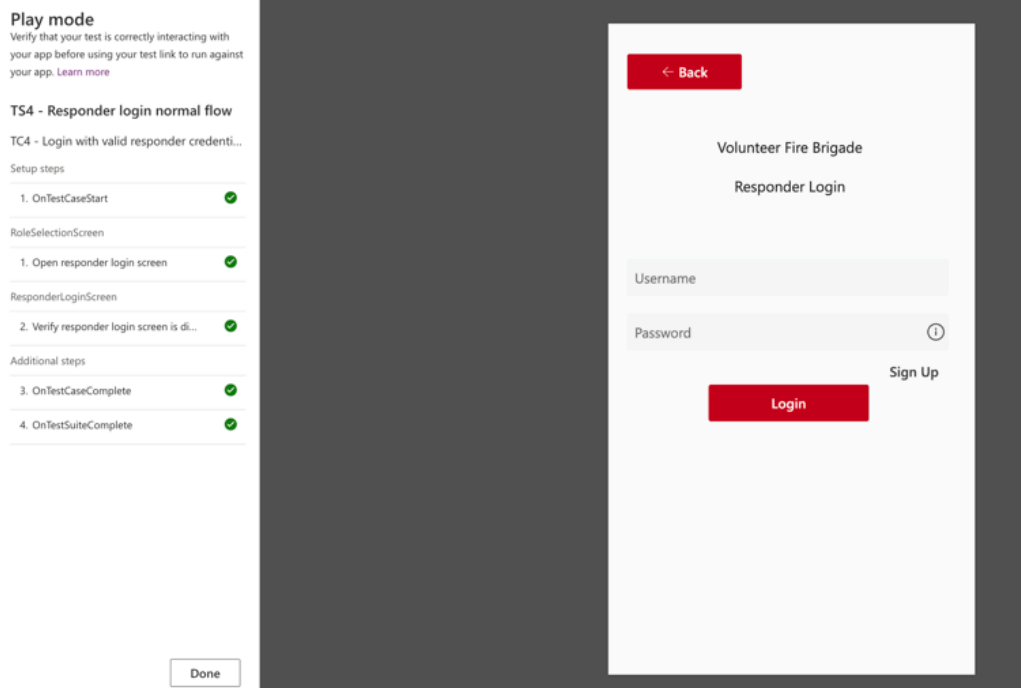


Figure TC4.2. Manual evidence showing the valid test responder account entered on the login screen. The password is hidden by the app input field.

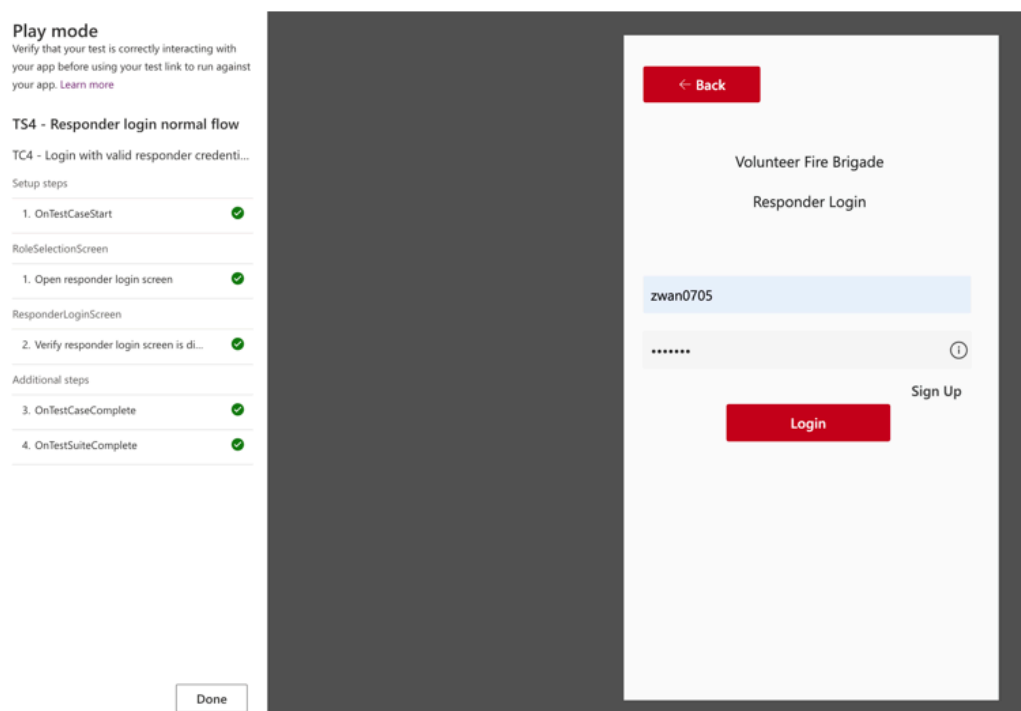
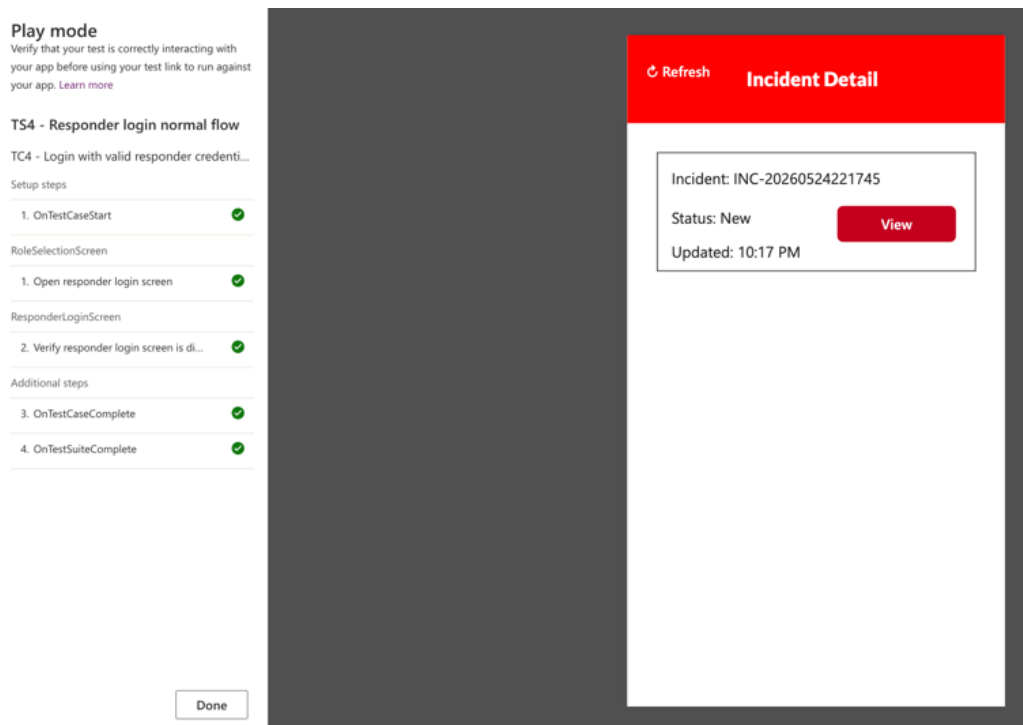


Figure TC4.3. Manual evidence showing that the responder successfully reached the incident workflow after login.



TC5 — Reject invalid responder login

Purpose:

This test verifies that an invalid responder username or password is rejected and does not allow access to the responder workflow.

Expected result:

The app should keep the user on the responder login screen when invalid credentials are entered. The user should not be able to access the incident overview or any responder-only workflow. Ideally, the app should also show a clear login error message.

Actual result:

Pass with documented workaround. Test Studio confirmed that the responder login screen opened correctly from the role selection page. Because Test Studio did not reliably enter values into the login input fields, the invalid credential attempt was verified manually. Manual testing confirmed that invalid credentials were rejected and the user could not proceed into the responder workflow.

Evidence:

Figure TC5.1. Automated Test Studio result showing that the responder login screen opens correctly.

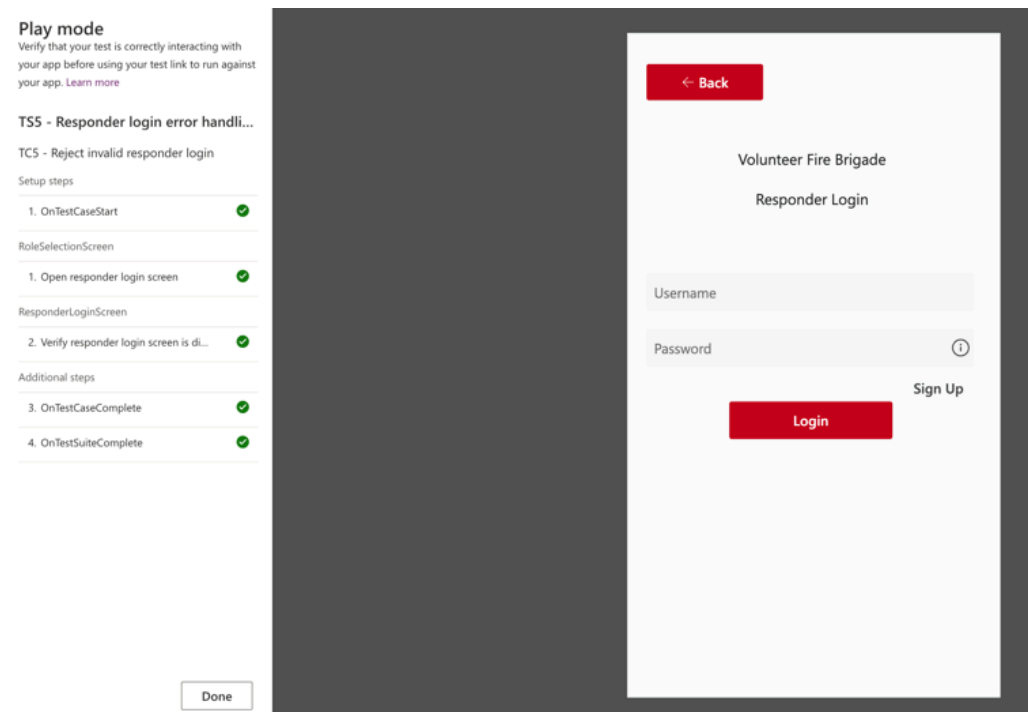
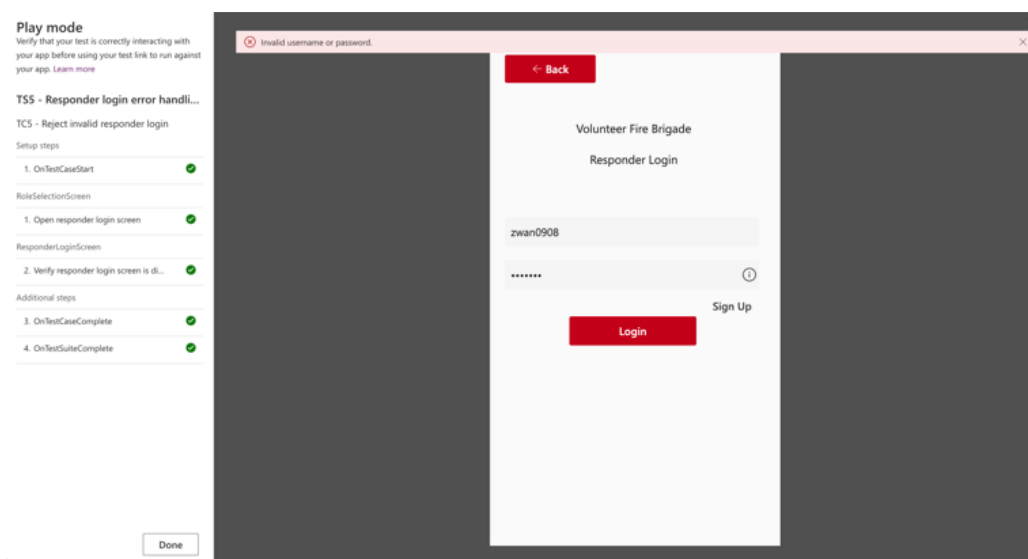


Figure TC5.2. Manual evidence showing that invalid responder credentials are rejected and the user remains on the login screen.



TC6 — Show active incidents in the incident overview queue

Purpose:

This test verifies that the responder incident overview screen displays active incident records that require responder attention. It also checks that completed or removed incidents are not shown in the active queue.

Expected result:

The app should open the incident overview screen and display active incident records in the responder queue. Records that have already been marked as Done or handled as false reports should not appear in the active queue.

Actual result:

Pass. Test Studio confirmed that the incident overview screen opened correctly and that the incident queue was displayed. The queue contained active incident records for responder action. Completed and false-report records were checked against the SharePoint data and were not shown in the active incident queue.

Evidence:

Figure TC6.1. Test Studio result showing that the incident overview screen opens and the active incident queue is displayed.

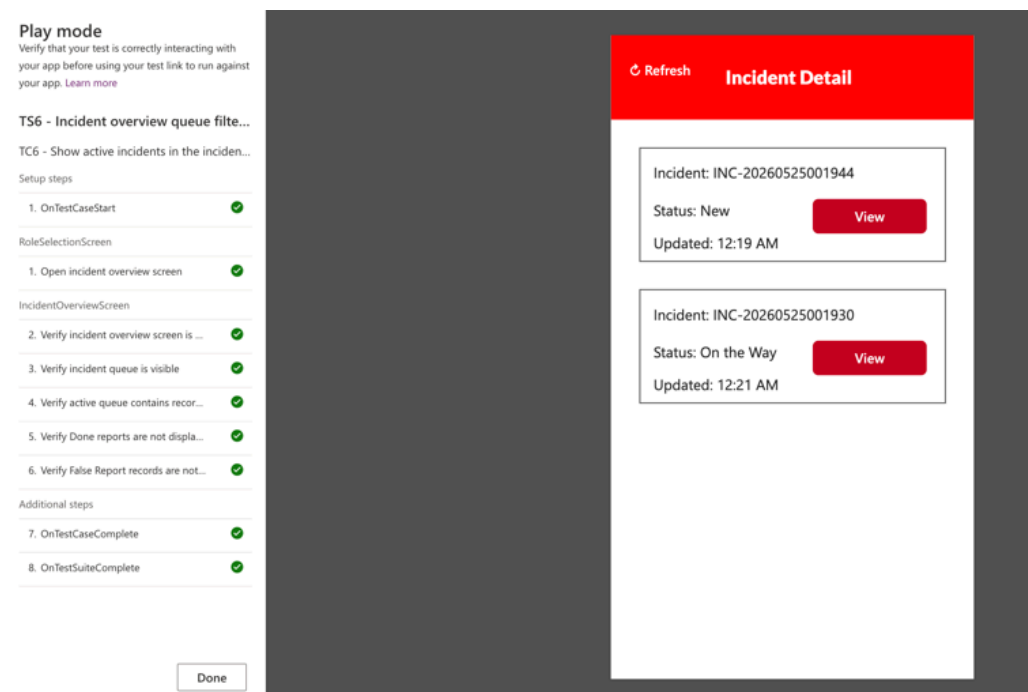


Figure TC6.2. SharePoint evidence used to compare active and non-active incident records during queue filtering verification.

Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From Sa...	Incident Type	Support Requested	Add column
INC-20260524211745	-33.887872, 151.198567	zwan0264@uni.syd...	Done	2 hours ago	About a minute ago	Captured	1	Structure Fire		
INC-20260525001909	-33.887926, 151.198505	zwan0264@uni.syd...	False Report	3 minutes ago	About a minute ago	Captured	2	Medical EmERGE...		
INC-20260525001930	-33.887926, 151.198505	zwan0264@uni.syd...	On the Way	3 minutes ago	About a minute ago	Captured	3	Vehicle Fire		
INC-20260525001944	-33.887926, 151.198505	zwan0264@uni.syd...	New	3 minutes ago	3 minutes ago	Captured	4	Other Emergency		

TC7 — Accept new incident and move to tracking

Purpose:

This test verifies that a responder can open a New incident, review the incident details, accept the dispatch request, and move into the dispatch tracking workflow.

Expected result:

The app should allow the responder to open a New incident, view the incident type, location and verification information, then accept the dispatch request. After acceptance, the app should navigate to the Dispatch Tracking screen and show the incident status as On the Way.

Actual result:

Pass with documented workaround. Test Studio verified the stable parts of the flow, including opening the incident overview screen and displaying active incident records. The selected incident was then followed through the responder detail screen and dispatch tracking screen. After the responder accepted the request, the app displayed the incident as On the Way, and the SharePoint backend showed the same incident record updated to On the Way.

Evidence:

Figure TC7.1. Test Studio opened the incident overview screen and showed a New incident available for responder action.

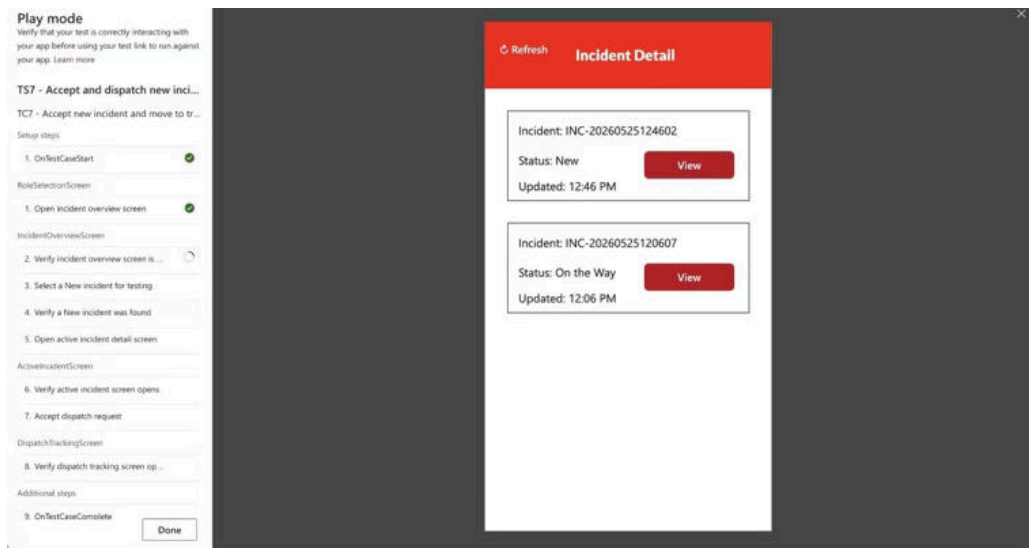


Figure TC7.2. Active incident detail screen before acceptance, showing the selected New incident, incident type, location, GPS status, and verification information.

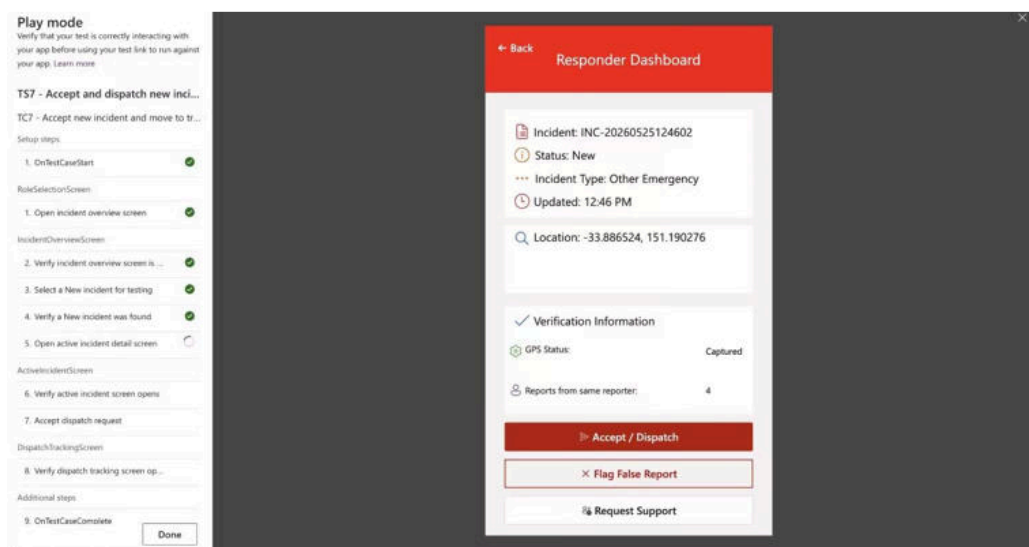
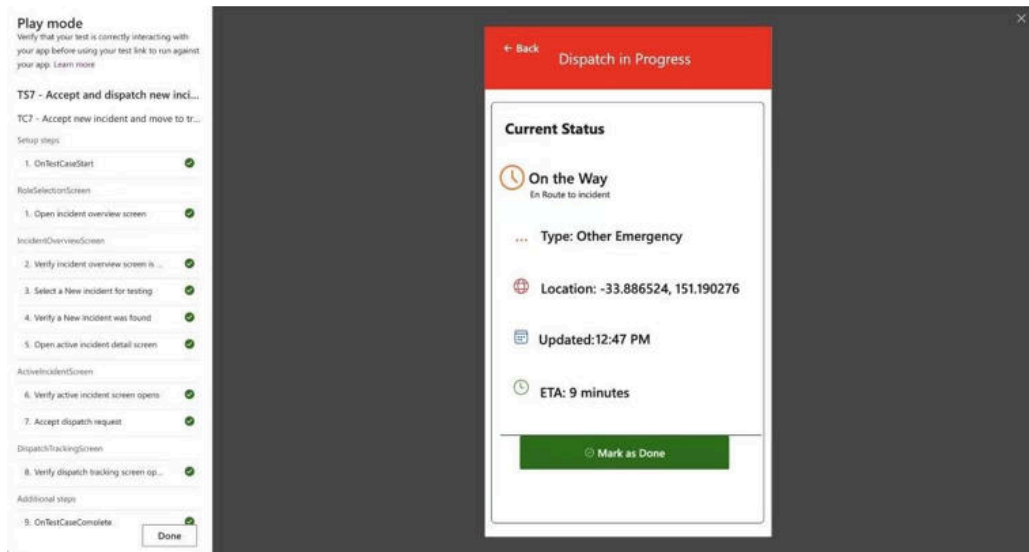


Figure TC7.3. Dispatch tracking screen after the responder accepted the incident, showing that the incident moved into the On the Way tracking state.



TC8 — Request additional support and show tracking warning

Purpose:

This test verifies that a responder can request additional support for an active incident and that the app clearly shows the support request on the dispatch tracking screen.

Expected result:

The responder should be able to open a New incident, request additional support, and move to the Dispatch Tracking screen. The tracking screen should show the incident as On the Way and display the Additional Support Requested warning.

Actual result:

Pass. The responder opened the selected New incident, requested additional support, and the app navigated to the Dispatch Tracking screen. The final screen showed the Additional Support Requested warning and kept the incident status as On the Way. The backend SharePoint record was also checked separately to confirm that the support flag was saved.

Evidence:

Figure TC8.1. Incident overview screen showing the selected New incident before the responder opens it for the support request test.

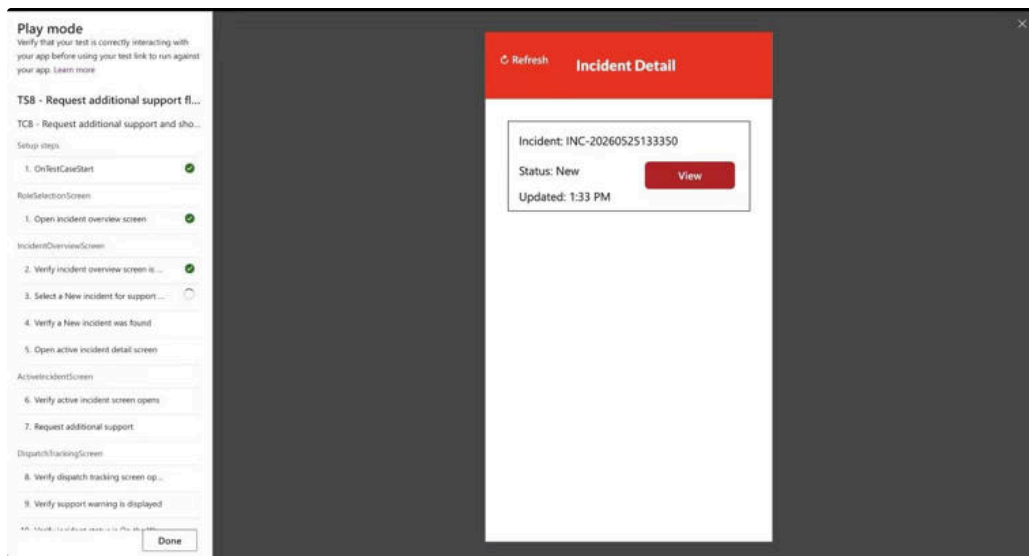


Figure TC8.2. Active incident detail screen before requesting support, showing the selected incident information and the Request Support button.

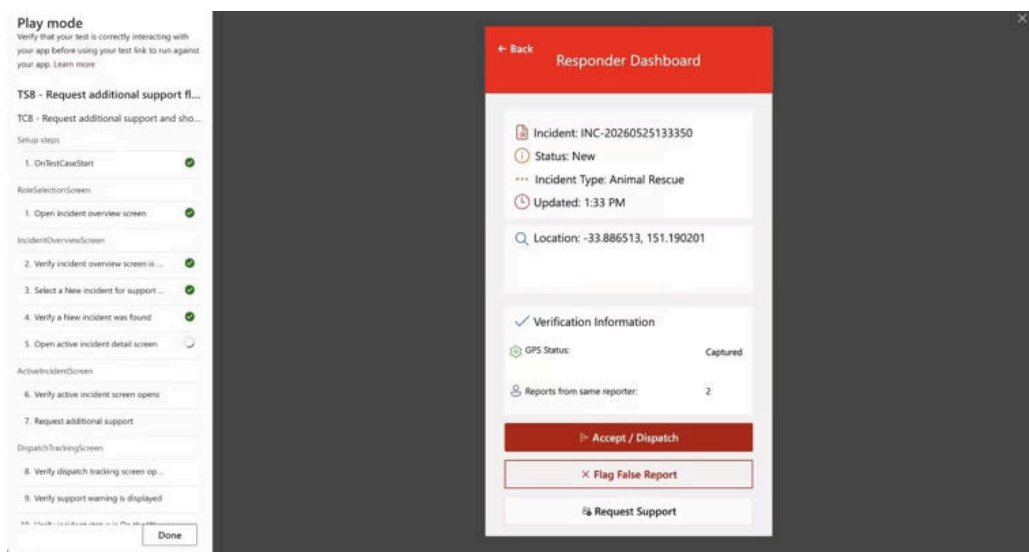
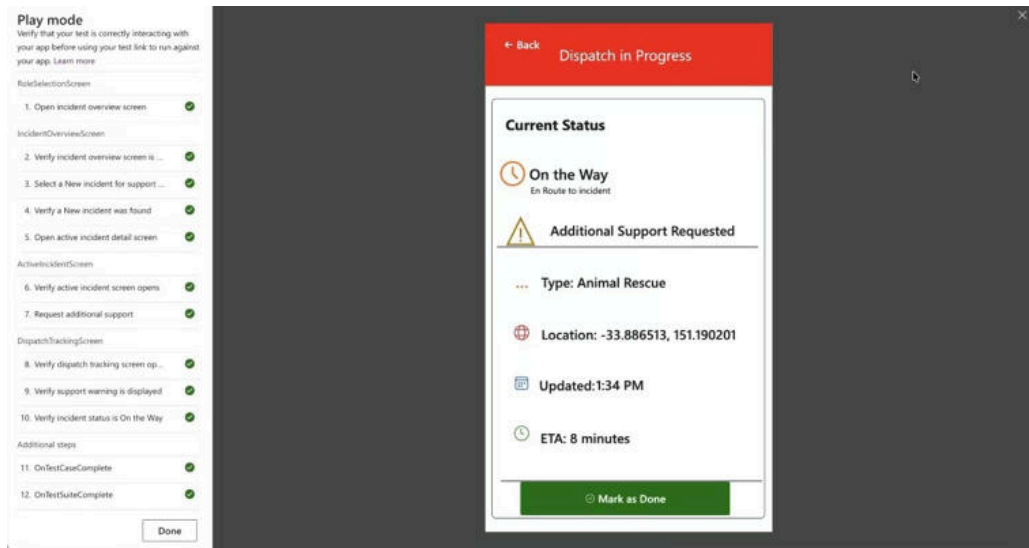


Figure TC8.3. Dispatch tracking screen after the responder requested additional support, showing the support warning and the On the Way status.



TC9 — Open On the Way incident directly in tracking screen

Purpose:

This test verifies that a responder can reopen an incident that is already marked as On the Way and access the Dispatch Tracking screen directly, without repeating the accept/dispatch step.

Expected result:

The app should identify an existing On the Way incident, open the Dispatch Tracking screen directly, and display the incident status, type, location, updated time, and ETA.

Actual result:

Pass. The test opened an existing On the Way incident and navigated directly to the Dispatch Tracking screen. The tracking screen displayed the On the Way status and the key incident details needed by the responder.

Evidence:

Figure TC9.1. Incident overview screen showing an existing On the Way incident selected for direct tracking.

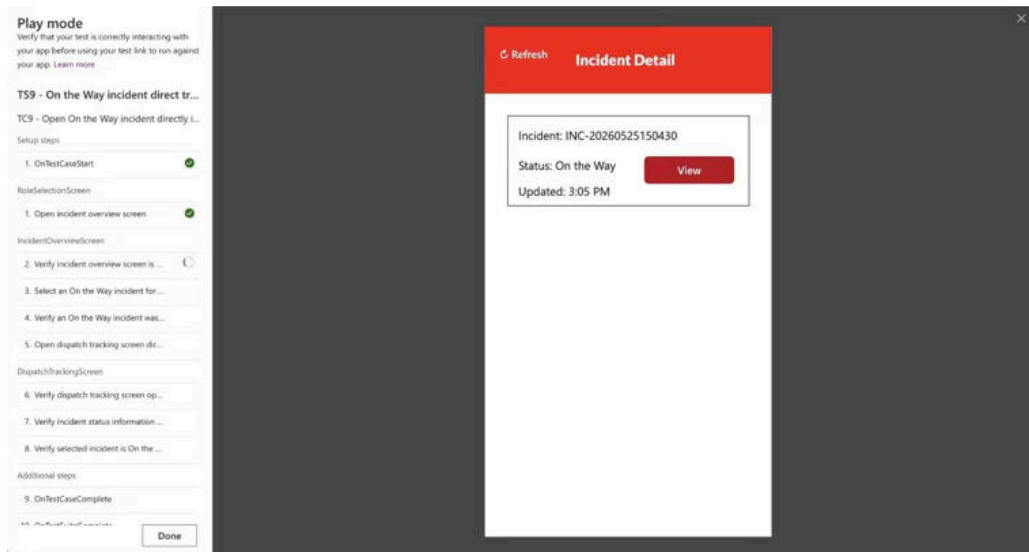
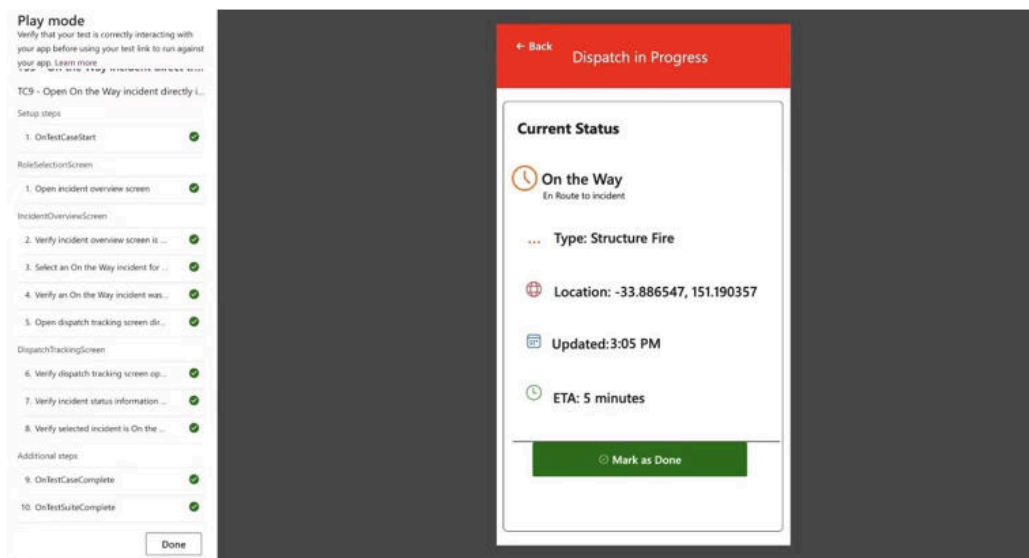


Figure TC9.2. Dispatch tracking screen opened directly for the selected On the Way incident.



TC10 — Flag incident as false report

Purpose:

This test verifies that a responder can open a New incident, review the verification information, flag the incident as a false report, and return to the incident overview without keeping that incident in the active queue.

Expected result:

The app should open the selected New incident, show the responder the incident details and verification information, allow the responder to press Flag False Report, and then return to the

incident overview screen. The flagged incident should no longer appear in the active incident queue.

Actual result:

Pass. The selected incident was opened successfully, and the verification information was visible before the false report action was taken. After the responder selected Flag False Report, the app returned to the incident overview screen and the selected incident was removed from the active queue. The backend SharePoint record was also checked separately and confirmed as updated to False Report.

Evidence:

Figure TC10-1 shows the incident detail screen before the false report action. The responder can see the incident status, location, GPS status, report count, and the Flag False Report button.

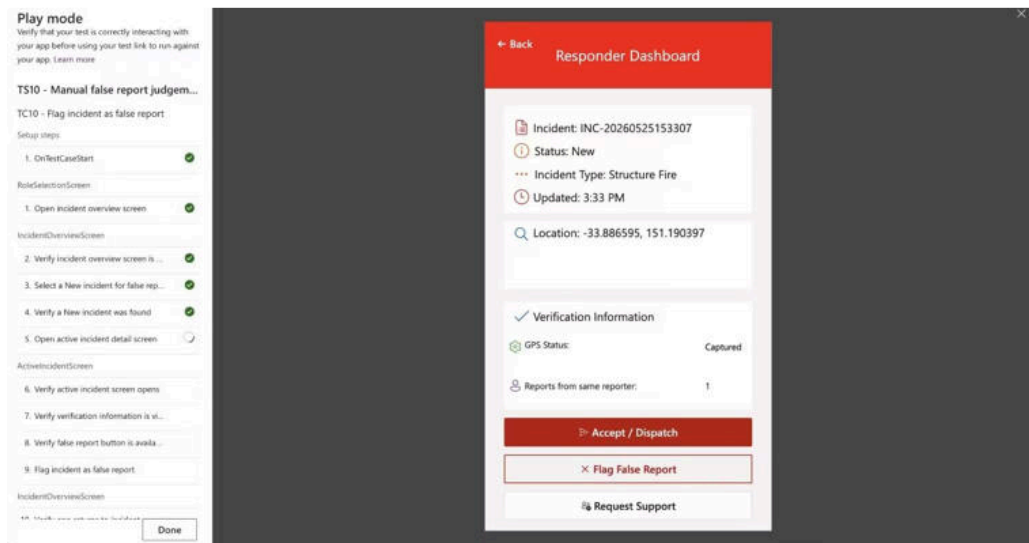
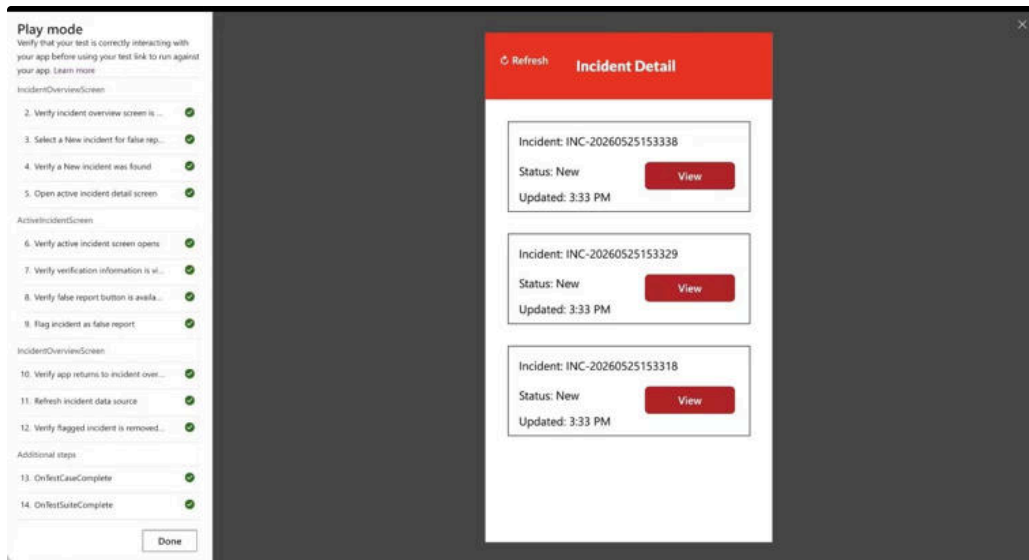


Figure TC10-2 shows the final Test Studio result after the action. The test passed, and the flagged incident was no longer displayed in the active queue.



TC11 — Mark incident as Done and remove it from active queue

Purpose:

This test verifies that a responder can complete an active dispatch after an incident has already been accepted and moved to On the Way.

Expected result:

The app should open the Dispatch Tracking screen for an On the Way incident, show the Mark as Done button, allow the responder to complete the incident, return to the Incident Overview screen, and remove the completed incident from the active queue.

Actual result:

Pass. The selected On the Way incident opened correctly in the Dispatch Tracking screen, and the Mark as Done button was available. After the responder marked the incident as Done, the app returned to the Incident Overview screen and the completed incident was no longer shown in the active incident queue.

Evidence:

Figure TC11.1. Dispatch tracking screen showing the selected On the Way incident before it is marked as Done.

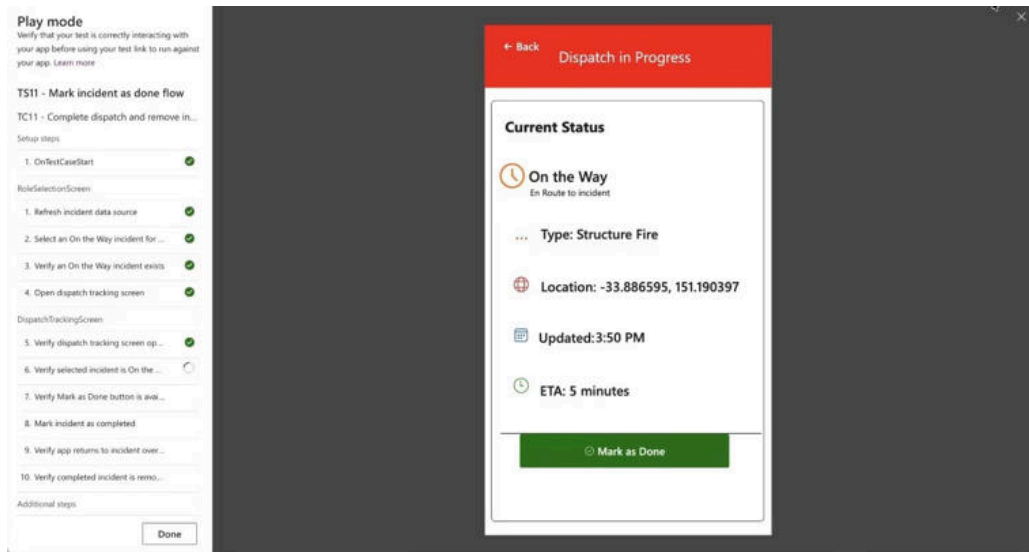
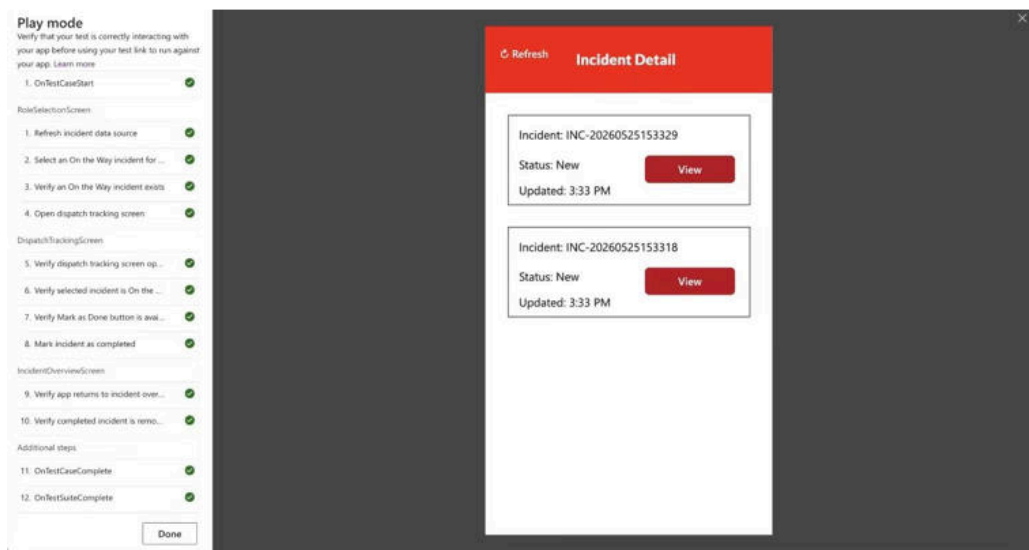


Figure TC11.2. Test Studio result after marking the incident as Done. The app returned to the incident overview and the completed incident was removed from the active queue.



Test Studio Limitations and Workarounds

L1 — Incident type dropdown could not be reliably replayed

Limitation:

During TC1, Power Apps Test Studio was able to replay the role selection and navigate to the emergency confirmation screen, but it did not reliably replay the modern dropdown used for selecting the incident type. Because the incident type was not selected during playback, the Confirm button stayed disabled and the final assertion for the Emergency Status screen failed.

Workaround:

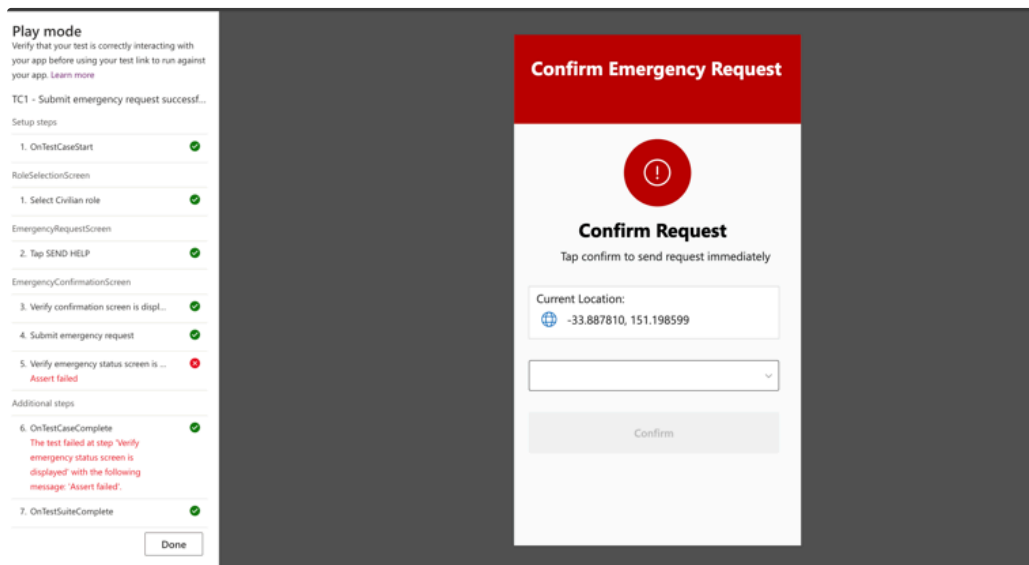
We kept the stable automated parts of TC1 in Test Studio, including selecting the Civilian role, opening the emergency request flow, navigating to the confirmation screen, and verifying that the GPS coordinates were displayed. The incident type selection and final Confirm action were then checked manually with screenshots, because these steps depended on the dropdown control that Test Studio did not handle consistently.

Result:

Pass with documented workaround. The failed Test Studio step was caused by the testing tool limitation, not by the app logic. The app behaved correctly by preventing submission when no incident type was selected. After selecting the incident type manually, the Confirm button became active and the request was successfully submitted to the Emergency Status / Safety Guidance screen.

Evidence:

Figure L1. Test Studio dropdown limitation: the incident type was not selected automatically during playback, so the Confirm button remained disabled and the final Emergency Status screen assertion failed.



L2 — Sign-up text input fields could not be reliably replayed

Limitation:

During TC3, Test Studio was able to open the responder sign-up page, but it did not reliably enter values into the sign-up text input fields. The test failed when trying to set the responder username field, even though the same form worked normally when tested manually in the app.

This appeared to be a Test Studio limitation with the input controls rather than a failure of the sign-up function itself.

Workaround:

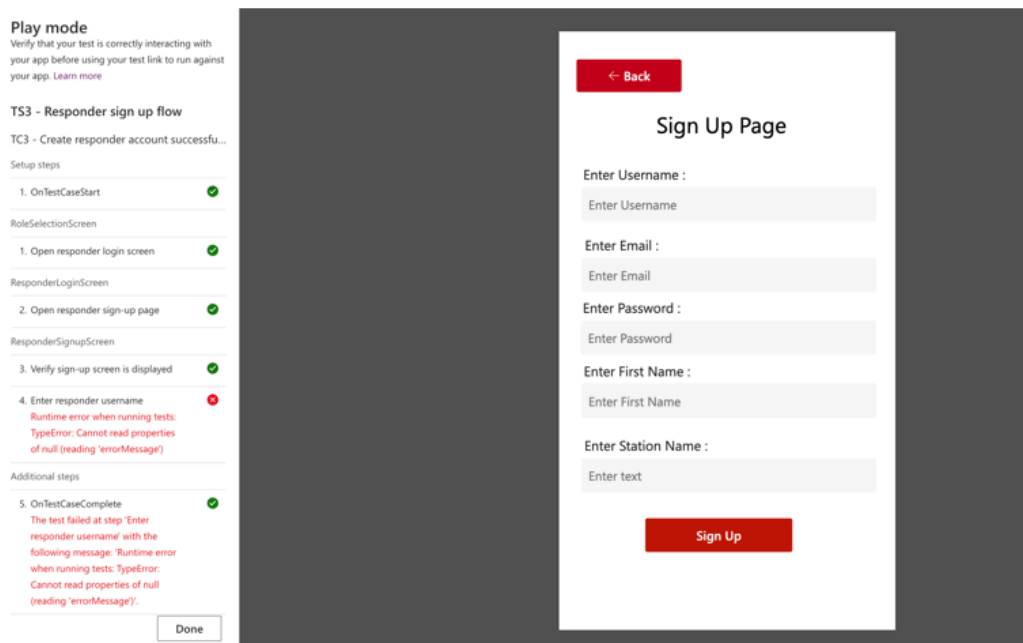
We kept the stable automated part in Test Studio, including opening the responder login screen, navigating to the sign-up page, and verifying that the sign-up page was displayed. The remaining form entry and account creation steps were completed manually and documented with screenshots.

Result:

Pass with documented workaround. The responder sign-up page was reachable through the app, and manual testing confirmed that a responder account could be created successfully.

Evidence:

Figure L2. Test Studio input-control limitation where the test failed when trying to enter the responder username automatically.



L3 — Login input fields could not be reliably replayed

Limitation:

During TC4 and TC5, Power Apps Test Studio was able to open the responder login screen, but it did not reliably enter values into the username and password input fields. The test failed when trying to set the login input values, even though the same login behaviours worked correctly when tested manually.

Workaround:

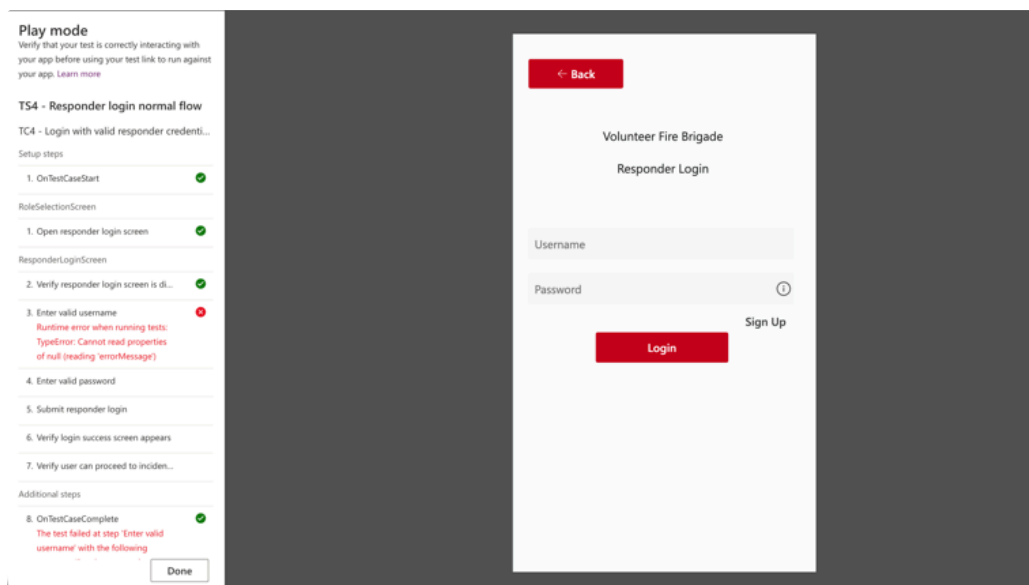
We kept the stable automated part of the login tests in Test Studio, including opening the responder login screen and verifying that the screen was displayed. Valid and invalid credential behaviours were then checked manually with screenshots.

Result:

Pass with documented workaround. The limitation was caused by Test Studio replaying the input controls inconsistently, not by the app login logic. Manual testing confirmed that valid credentials allow access and invalid credentials are rejected.

Evidence:

Figure L3. Test Studio input-field replay limitation during the responder login test.



L4 — Gallery item selection could not be replayed reliably

Limitation:

During TC7, Power Apps Test Studio sometimes failed when replaying the View button inside the incident gallery. The app itself displayed the incident overview screen correctly, but Test Studio did not always select the gallery item reliably and returned a runtime error during playback.

Workaround:

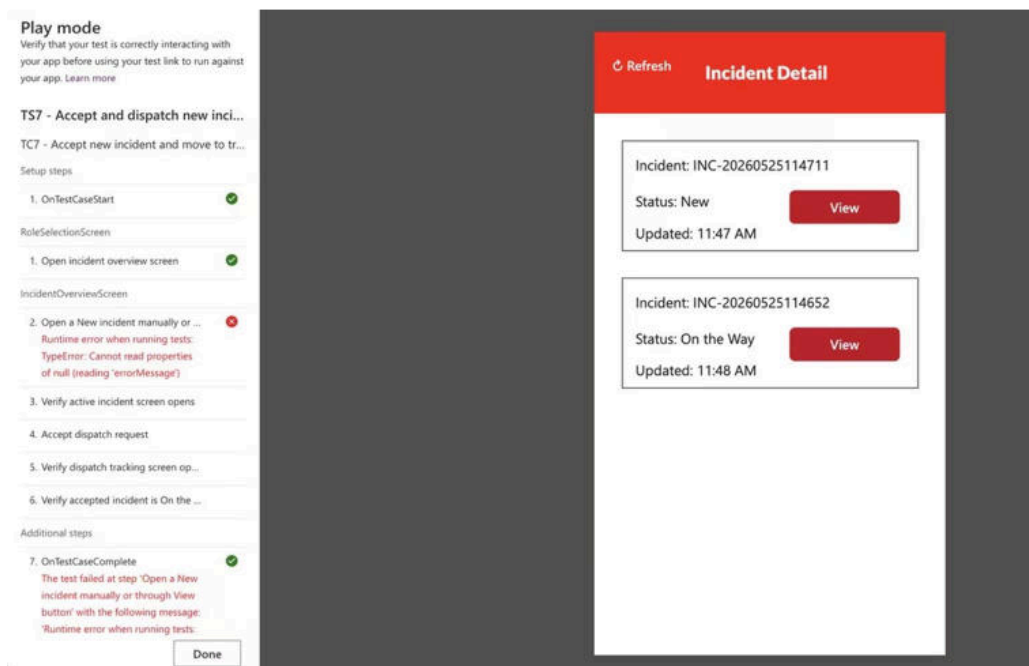
To avoid treating a Test Studio playback issue as an app failure, I kept the stable automated steps in Test Studio, including opening the incident overview screen and checking that active incidents were displayed. I then used a consistent manual evidence chain for the selected incident, including the active incident detail screen, dispatch tracking screen, and SharePoint backend update.

Result:

Pass with documented workaround. The failed replay was caused by Test Studio's limitation with repeated gallery controls, not by the incident dispatch workflow. The same workflow was verified using screenshots from the app and SharePoint for INC-20260525124602.

Evidence:

Figure L4. Test Studio playback issue when selecting a repeated View button inside the incident gallery.



Manual Backend evidence

The automated Test Studio cases mainly verify the app screens, navigation, and visible user interactions. Some backend behaviours were checked manually because they depend on SharePoint data updates and are easier to verify directly in the connected lists. For these checks, we compared the Power Apps action with the corresponding SharePoint record to confirm that the backend data was created or updated correctly.

BE1 — New incident record created in SharePoint

Purpose:

This backend check verifies that a submitted civilian emergency request is saved into the SharePoint EmergencyIncidents list.

Expected result:

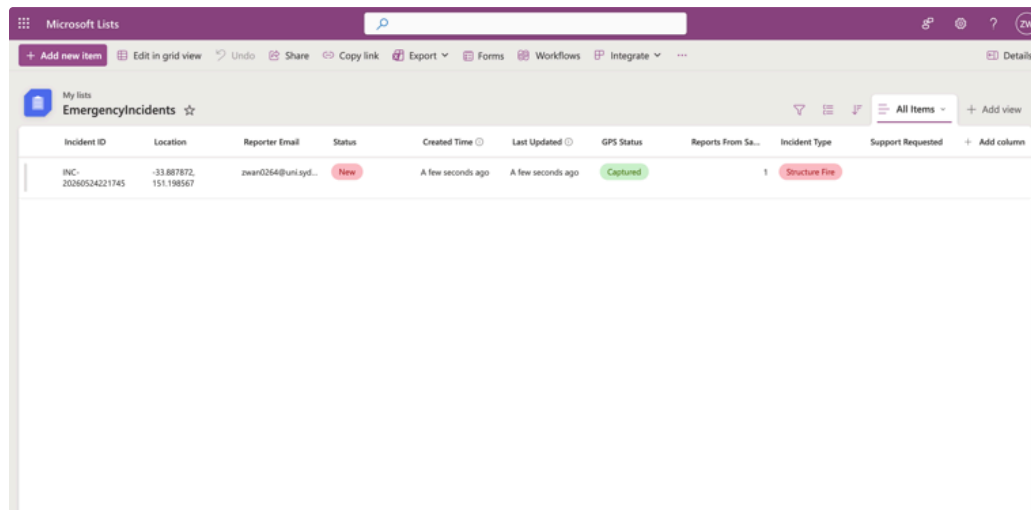
A new incident record should appear in SharePoint with an Incident ID, location, reporter email, status, GPS status, report count, and incident type.

Actual result:

Pass. After submitting the request from Power Apps, a new SharePoint record was created. The record stored the generated Incident ID, GPS coordinates, reporter email, New status, Captured GPS status, report count, and selected incident type.

Evidence:

Figure BE1.New incident record created in SharePoint after civilian submission.



The screenshot shows a Microsoft Lists interface for a list named 'EmergencyIncidents'. The list contains one item with the following details:

Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From Sa...	Incident Type	Support Requested	+ Add column
INC-20260524221745	-33.887872, 151.198567	zwan0264@uni.syd...	New	A few seconds ago	A few seconds ago	Captured	1	Structure Fire		

BE2 — Responder account record stored in SharePoint**Purpose:**

This backend check verifies that a responder account created through the app is saved into the ResponderDetails SharePoint list.

Expected result:

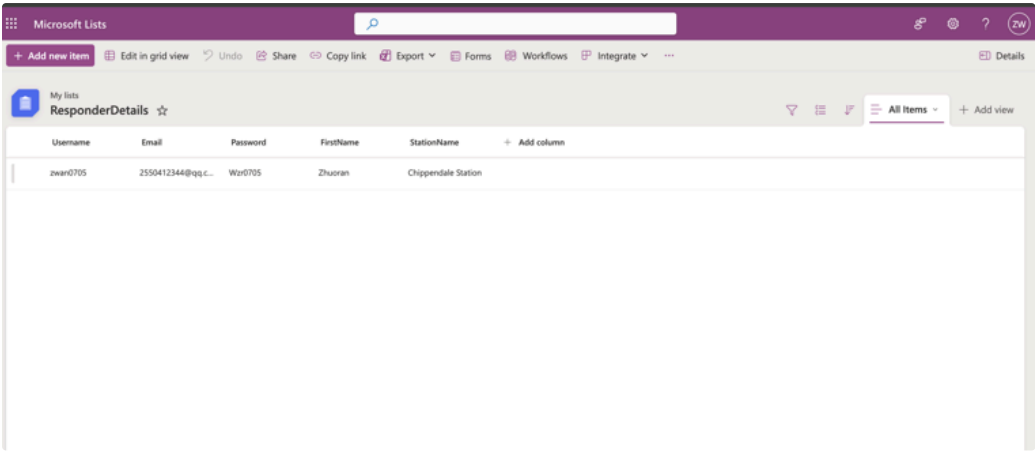
After a responder completes the sign-up form, a new record should appear in the ResponderDetails list with the username, email, password, first name, and station name fields stored correctly.

Actual result:

Pass. After the responder account was created in Power Apps, a matching record appeared in the ResponderDetails SharePoint list. The stored username, email, password, first name, and station name matched the details entered during the manual sign-up test.

Evidence:

Figure BE2.ResponderDetails SharePoint record created after the responder sign-up form was submitted.



BE3 — New incident record created in SharePoint

Purpose:

This backend check verifies that a civilian emergency request is saved into the SharePoint EmergencyIncidents list after submission from Power Apps.

Expected result:

A new incident record should appear in SharePoint with an Incident ID, captured location, reporter email, New status, GPS status, report count, and selected incident type.

Actual result:

Pass. After the request was submitted, SharePoint created a new record for INC-20260525124602. The record stored the GPS coordinates, reporter email, New status, Captured GPS status, report count, and the selected incident type.

Evidence:

Figure BE3. New incident record created in SharePoint after civilian submission.

The screenshot shows a Microsoft Lists interface for a list named 'EmergencyIncidents'. The table contains four incident records. The first three records have a 'Status' of 'Done', 'False Report', and 'On the Way' respectively. The fourth record, with Incident ID 'INC-20260525124602', has a 'Status' of 'New'. The 'Created Time' and 'Last Updated' columns show timestamps for each record. The 'GPS Status' column shows 'Captured' for all records. The 'Incident Type' column shows 'Structure Fire', 'Medical Emergency', 'Vehicle Fire', and 'Other Emergency' respectively.

Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From SA...	Incident Type	Support Requested	Add column
INC-20260525120546	-33.886534, 151.190306	zwan0264@uni.syd...	Done	40 minutes ago	39 minutes ago	Captured	1	Structure Fire		
INC-20260525120557	-33.886534, 151.190306	zwan0264@uni.syd...	False Report	40 minutes ago	39 minutes ago	Captured	2	Medical Emergency		
INC-20260525120607	-33.886534, 151.190306	zwan0264@uni.syd...	On the Way	40 minutes ago	39 minutes ago	Captured	3	Vehicle Fire		
INC-20260525124602	-33.886524, 151.190276	zwan0264@uni.syd...	New	A few seconds ago	A few seconds ago	Captured	4	Other Emergency		

BE4 — Incident status updated after responder acceptance

Purpose:

This backend check verifies that responder actions update the same incident record in SharePoint, rather than only changing the front-end screen.

Expected result:

After the responder accepts the dispatch request, the same incident record should remain in SharePoint and its status should update from New to On the Way.

Actual result:

Pass. The record for INC-20260525124602 was updated from New to On the Way after the responder accepted the dispatch request. This confirms that the responder workflow is connected to the SharePoint backend.

Evidence:

Figure BE4. SharePoint EmergencyIncidents list showing the same incident updated to On the Way after responder acceptance.

The screenshot shows the same Microsoft Lists interface for 'EmergencyIncidents'. The table now shows four incident records. The first three records have a 'Status' of 'Done', 'False Report', and 'On the Way' respectively. The fourth record, with Incident ID 'INC-20260525124602', has a 'Status' of 'On the Way'. The 'Created Time' and 'Last Updated' columns show timestamps for each record. The 'GPS Status' column shows 'Captured' for all records. The 'Incident Type' column shows 'Structure Fire', 'Medical Emergency', 'Vehicle Fire', and 'Other Emergency' respectively.

Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From SA...	Incident Type	Support Requested	Add column
INC-20260525120546	-33.886534, 151.190306	zwan0264@uni.syd...	Done	42 minutes ago	40 minutes ago	Captured	1	Structure Fire		
INC-20260525120557	-33.886534, 151.190306	zwan0264@uni.syd...	False Report	42 minutes ago	41 minutes ago	Captured	2	Medical Emergency		
INC-20260525120607	-33.886534, 151.190306	zwan0264@uni.syd...	On the Way	42 minutes ago	41 minutes ago	Captured	3	Vehicle Fire		
INC-20260525124602	-33.886524, 151.190276	zwan0264@uni.syd...	On the Way	About a minute ago	A few seconds ago	Captured	4	Other Emergency		

BE5 — Support request flag saved in SharePoint

Purpose:

This backend check verifies that the support request is saved in SharePoint, not only displayed on the Power Apps screen.

Expected result:

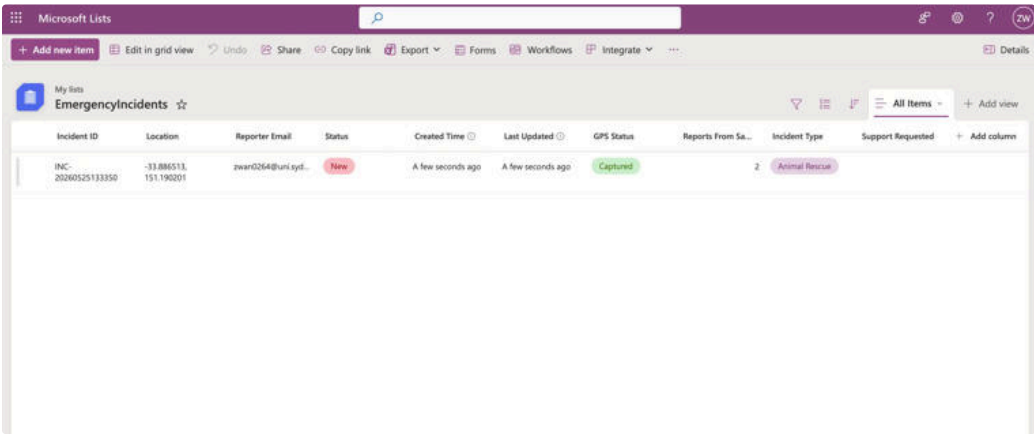
Before the support request, the selected incident should have New status and no support flag. After the responder requests support, the same incident record should show Support Requested as true and the status should be updated to On the Way.

Actual result:

Pass. The SharePoint record for the same incident was updated correctly. Before the support request, the record showed New status and no support flag. After the request, the same record showed On the Way status and the Support Requested field was ticked.

Evidence:

Figure BE5.1. SharePoint record before the support request, showing the selected incident with New status and no support flag.



Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From Sa...	Incident Type	Support Requested	Add column
INC-20240525133350	-33.886213, 151.196201	zwan0264@uni.syd...	New	A few seconds ago	A few seconds ago	Captured	2	Animal Rescue		

Figure BE5.2. SharePoint record after the support request, showing that the incident status was updated to On the Way and the Support Requested field was saved.

Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From Sa...	Incident Type	Support Requested	+ Add column
INC-20260525133350	-33.886511, 151.190281	zwan0264@uni Syd...	On the way	4 minutes ago	3 minutes ago	Captured	2	Animal Rescue	✓	

BE6 — False report status saved in SharePoint

Purpose:

This backend check verifies that when a responder flags an incident as a false report, the same incident record is updated in the SharePoint EmergencyIncidents list.

Expected result:

Before the false report action, the selected incident should have New status. After the responder selects Flag False Report, the same incident record should show False Report status and should no longer appear in the active incident queue.

Actual result:

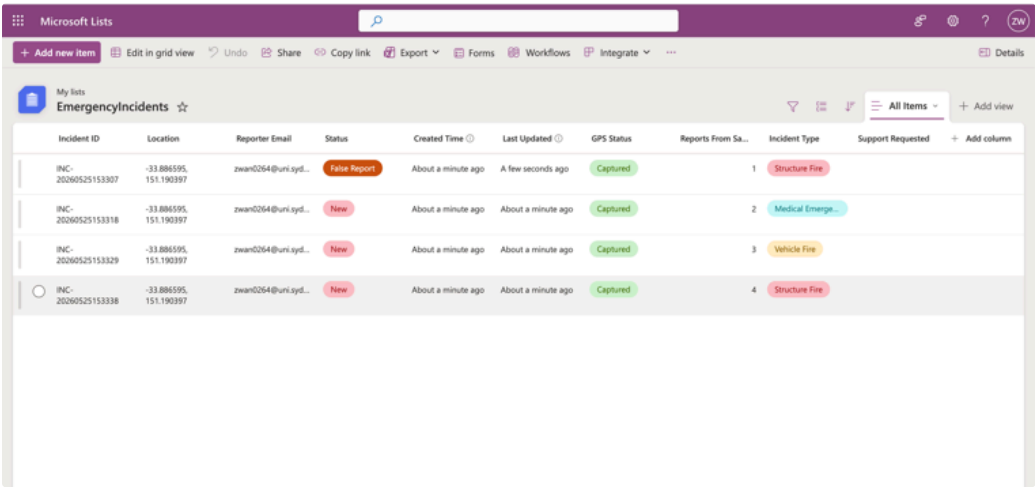
Pass. The selected SharePoint record was updated correctly after the responder flagged it as a false report. The record changed from New to False Report, and the incident was removed from the active queue in Power Apps. This confirms that the false report action is saved in the backend.

Evidence:

Figure BE6.1. SharePoint record before the false report action, showing the selected incident with New status.

Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From Sa...	Incident Type	Support Requested	+ Add column
INC-20260525153307	-33.886595, 151.190397	zwan0264@uni Syd...	New	About a minute ago	About a minute ago	Captured	1	Structure Fire		
INC-20260525153318	-33.886595, 151.190397	zwan0264@uni Syd...	New	A few seconds ago	A few seconds ago	Captured	2	Medical Emergency		
INC-20260525153329	-33.886595, 151.190397	zwan0264@uni Syd...	New	A few seconds ago	A few seconds ago	Captured	3	Vehicle Fire		
INC-20260525153338	-33.886595, 151.190397	zwan0264@uni Syd...	New	A few seconds ago	A few seconds ago	Captured	4	Structure Fire		

Figure BE6.2. SharePoint record after the false report action, showing the same incident updated to False Report.



Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From Sa...	Incident Type	Support Requested	Add column
INC-20200525153307	-33.886595, 151.190397	zwan0264@buni.syd...	False Report	About a minute ago	A few seconds ago	Captured	1	Structure Fire		
INC-20200525153318	-33.886595, 151.190397	zwan0264@buni.syd...	New	About a minute ago	About a minute ago	Captured	2	Medical Emergency		
INC-20200525153329	-33.886595, 151.190397	zwan0264@buni.syd...	New	About a minute ago	About a minute ago	Captured	3	Vehicle Fire		
INC-20200525153338	-33.886595, 151.190397	zwan0264@buni.syd...	New	About a minute ago	About a minute ago	Captured	4	Structure Fire		

BE7 — Completed incident status saved in SharePoint

Purpose:

This backend check verifies that when a responder marks an active dispatch as completed, the same incident record is updated in the SharePoint EmergencyIncidents list.

Expected result:

Before completion, the selected incident should have On the Way status. After the responder selects Mark as Done, the same SharePoint record should show Done status and should no longer appear in the active incident queue.

Actual result:

Pass. The SharePoint record for the same incident was updated correctly. Before completion, the incident showed On the Way status. After the responder selected Mark as Done in Power Apps, the same incident record showed Done in SharePoint. The completed incident was also removed from the active queue in the app.

Evidence:

Figure BE7.1. SharePoint record before completion, showing the selected incident with On the Way status.

Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From Sa...	Incident Type	Support Requested	Add column
INC-20260525153318	-33.886595, 151.190397	zwan0264@uni.syd...	New	17 minutes ago	17 minutes ago	Captured	2	Medical EmERGE...		
INC-20260525153329	-33.886595, 151.190397	zwan0264@uni.syd...	New	17 minutes ago	17 minutes ago	Captured	3	Vehicle Fire		
INC-20260525153338	-33.886595, 151.190397	zwan0264@uni.syd...	On the Way	17 minutes ago	A few seconds ago	Captured	4	Structure Fire		

Figure BE7.2. SharePoint record after the responder selected Mark as Done, showing the same incident updated to Done.

Incident ID	Location	Reporter Email	Status	Created Time	Last Updated	GPS Status	Reports From Sa...	Incident Type	Support Requested	Add column
INC-20260525153318	-33.886595, 151.190397	zwan0264@uni.syd...	New	24 minutes ago	24 minutes ago	Captured	2	Medical EmERGE...		
INC-20260525153329	-33.886595, 151.190397	zwan0264@uni.syd...	New	23 minutes ago	23 minutes ago	Captured	3	Vehicle Fire		
INC-20260525153338	-33.886595, 151.190397	zwan0264@uni.syd...	Done	23 minutes ago	About a minute ago	Captured	4	Structure Fire		

Performance Evaluation using Power Apps Monitor

This section records three performance observations from Power Apps Monitor. We used Monitor to check the main data-heavy parts of the app: creating a civilian incident, loading incidents on the responder side, and updating an incident after a responder action.

Before each Monitor check, we cleared the Monitor messages so that the evidence only showed the workflow being tested. We focused mainly on network calls, status codes, duration, and whether the app showed any warnings that could affect performance or scalability.

PF1 — Civilian incident submission

Purpose:

This Monitor check observes the civilian emergency request flow, especially the point where a submitted request is written into the EmergencyIncidents SharePoint list.

Observation:

After the civilian submitted an emergency request, Monitor recorded a successful patch/create

operation. The createRow request returned status 201, which means the new incident record was created successfully in SharePoint. Monitor also showed several getRows and getLookupResults calls with status 200, meaning the app was able to read incident data back from the data source after submission.

The main createRow call took 482 ms, while later getRows calls were around 141–243 ms. This was acceptable for the prototype because the user still reached the Emergency Status / Safety Guidance screen without a blocking delay.

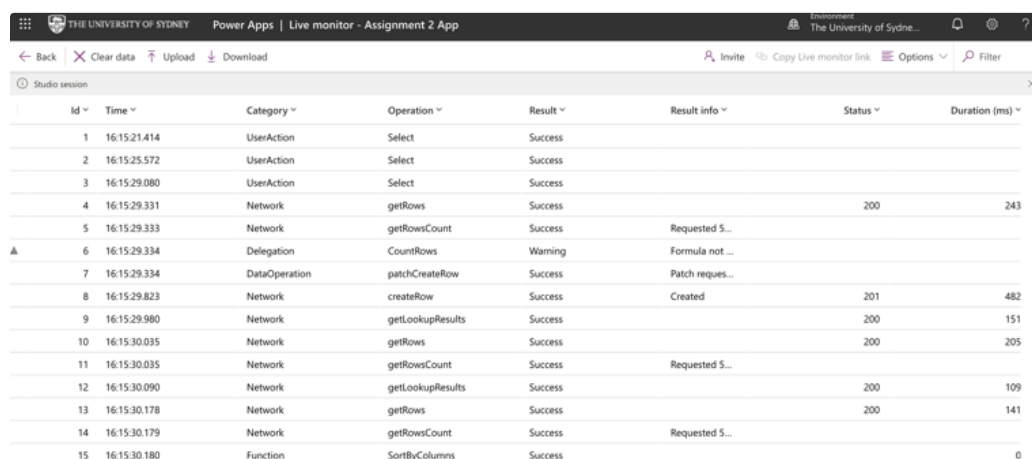
One delegation warning appeared for CountRows. This did not break the flow, but it is a useful finding because CountRows may become less reliable or slower with a larger SharePoint list. For this prototype dataset, the warning was noted as a scalability limitation rather than a functional failure.

Result:

Pass. The civilian request was created successfully, and Monitor showed successful SharePoint create and read operations.

Evidence:

Figure PF1. Power Apps Monitor result for civilian incident submission, showing patchCreateRow, createRow with status 201, and successful SharePoint read calls.



The screenshot shows the Power Apps Monitor interface for 'Assignment 2 App'. The table displays the following data:

	Id	Time	Category	Operation	Result	Result info	Status	Duration (ms)
	1	16:15:21.414	UserAction	Select	Success			
	2	16:15:25.572	UserAction	Select	Success			
	3	16:15:29.080	UserAction	Select	Success			
	4	16:15:29.331	Network	getRows	Success		200	243
	5	16:15:29.333	Network	getRowsCount	Success	Requested 5...		
▲	6	16:15:29.334	Delegation	CountRows	Warning	Formula not ...		
	7	16:15:29.334	DataOperation	patchCreateRow	Success	Patch reques...		
	8	16:15:29.823	Network	createRow	Success	Created	201	482
	9	16:15:29.980	Network	getLookupResults	Success		200	151
	10	16:15:30.035	Network	getRows	Success		200	205
	11	16:15:30.035	Network	getRowsCount	Success	Requested 5...		
	12	16:15:30.090	Network	getLookupResults	Success		200	109
	13	16:15:30.178	Network	getRows	Success		200	141
	14	16:15:30.179	Network	getRowsCount	Success	Requested 5...		
	15	16:15:30.180	Function	SortByColumns	Success			0

Figure PF1. Monitor evidence for civilian incident submission. The createRow request returned status 201, confirming that the emergency request was created in SharePoint.

PF2 — Responder incident overview refresh

Purpose:

This Monitor check observes how the responder side loads incident records from the

EmergencyIncidents SharePoint list.

Observation:

After opening the responder-side incident overview, Monitor recorded successful getRows and getRowCount operations. The getRows request returned status 200, which shows that the app was reading live incident data from SharePoint rather than using static screen content.

The first getRows call took 3334 ms, which was noticeably slower than the later calls in the same workflow. After the initial load, later data calls were much faster, with getLookupResults around 153 ms and getRows around 154 ms. This suggests that the first responder-side load is the heaviest part of the workflow, most likely because the app is loading the SharePoint list and preparing the gallery data.

Result:

Pass. The responder overview loaded incident data successfully. The main performance issue observed was the slower first getRows call, which should be improved in a future version if the SharePoint list becomes larger.

Evidence:

Figure PF2. Power Apps Monitor result for responder incident overview loading, showing successful getRows calls and the slower initial read request.

Id	Time	Category	Operation	Result	Result info	Status	Duration (ms)
1	16:24:11.778	UserAction	Select	Success			
2	16:24:24.240	UserAction	Select	Success			
3	16:24:27.586	Network	getRows	Success		200	3,334
4	16:24:27.588	Network	getRowCount	Success	Requested 1 ...		
5	16:24:30.749	UserAction	Select	Success			
6	16:24:30.911	Network	getLookupResults	Success		200	153
7	16:24:30.914	Network	getRows	Success		200	154
8	16:24:30.914	Network	getRowCount	Success	Requested 5...		
9	16:24:30.915	Function	SortByColumns	Success			0

Figure PF2. Monitor evidence for responder incident overview refresh. The first getRows call returned status 200 but took 3334 ms, making it the clearest performance observation in this section.

PF3 — Responder incident update

Purpose:

This Monitor check observes a responder action that updates an existing incident record. I used

the Request Support / incident update flow because it changes the incident data instead of only moving between screens.

Observation:

After the responder selected an incident and performed the update action, Monitor recorded a successful patchExistingRow data operation and a patchRow network request. The patchRow request returned status 200 and took 420 ms, which shows that the existing SharePoint record was updated successfully.

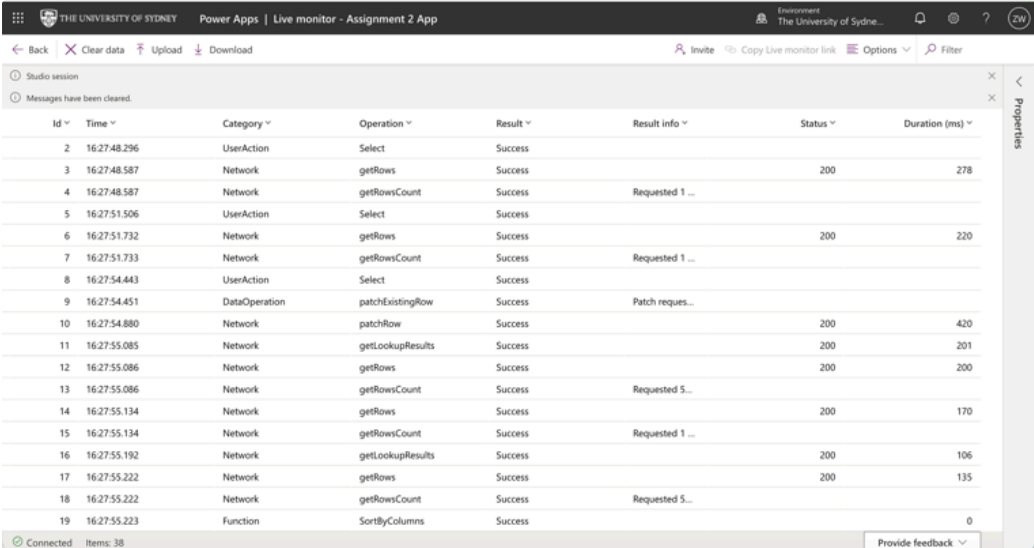
After the update, Monitor also recorded several successful getRows and getLookupResults calls with status 200. These calls were used to reload the updated incident data and refresh the responder-side display. Most of these follow-up calls were between about 106 ms and 278 ms, which was acceptable for this prototype.

Result:

Pass. The responder update was completed successfully. Monitor showed that the app updated the existing incident record and then refreshed the data shown in the app.

Evidence:

Figure PF3. Power Apps Monitor result for responder incident update, showing patchExistingRow, patchRow with status 200, and successful follow-up read operations.



Id	Time	Category	Operation	Result	Result info	Status	Duration (ms)
2	16:27:48.296	UserAction	Select	Success			
3	16:27:48.587	Network	getRows	Success		200	278
4	16:27:48.587	Network	getRowCount	Success	Requested 1 ...		
5	16:27:51.506	UserAction	Select	Success			
6	16:27:51.732	Network	getRows	Success		200	220
7	16:27:51.733	Network	getRowCount	Success	Requested 1 ...		
8	16:27:54.443	UserAction	Select	Success			
9	16:27:54.451	DataOperation	patchExistingRow	Success	Patch reques...		
10	16:27:54.880	Network	patchRow	Success		200	420
11	16:27:55.085	Network	getLookupResults	Success		200	201
12	16:27:55.086	Network	getRows	Success		200	200
13	16:27:55.086	Network	getRowCount	Success	Requested 5...		
14	16:27:55.134	Network	getRows	Success		200	170
15	16:27:55.134	Network	getRowCount	Success	Requested 1 ...		
16	16:27:55.192	Network	getLookupResults	Success		200	106
17	16:27:55.222	Network	getRows	Success		200	135
18	16:27:55.222	Network	getRowCount	Success	Requested 5...		
19	16:27:55.223	Function	SortByColumns	Success			0

Figure PF3. Monitor evidence for responder incident update. The patchRow request returned status 200 and took 420 ms, confirming that the existing incident was updated through SharePoint.

Performance summary:

Overall, the app performed reliably during the monitored workflows. The civilian submission created a new SharePoint record successfully, the responder overview loaded live incident data, and the responder update flow patched an existing record successfully.

The main performance issue found was the slower first `getRows` call on the responder overview screen, which took 3334 ms. This is acceptable for the current prototype, but it suggests that future versions should reduce the amount of data loaded into the incident gallery, use more specific filters, and avoid unnecessary `CountRows` operations where possible. The `CountRows` delegation warning was also documented because it may become more important when the SharePoint list contains many incident records.

Accessibility Evaluation

AE means “Accessibility Evaluation”. Each AE item checks one accessibility aspect of the app, including readability, navigation, input clarity, and platform limitations in Power Apps.

AE1 — Readability and visual contrast

Observation:

The main emergency screens use a simple red-and-white layout with large headings and clear action buttons. This helps users quickly identify the main action, such as SEND HELP, Confirm, Accept / Dispatch, Request Support, and Mark as Done. The most important buttons are visually separated from normal text, which reduces confusion in a high-pressure emergency situation.

Limitation:

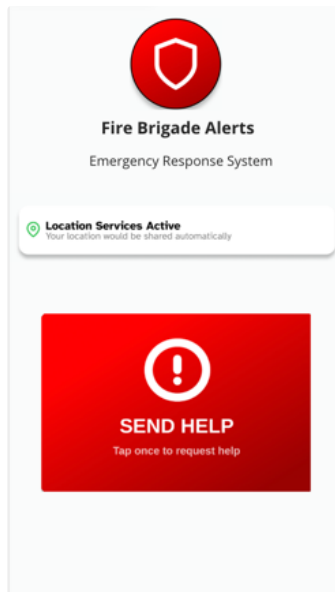
Some secondary information, such as GPS coordinates and update times, is smaller than the main headings. This is acceptable for the prototype, but a future version should increase the size of critical status text and avoid relying only on colour to communicate status.

Result:

Pass with improvement noted. The main workflow is readable, but some secondary information could be made larger for field use.

Evidence:

Figure AE1. Example screen showing the app’s red-and-white layout, large heading, and clear primary action button.



AE2 — Navigation and workflow simplicity

Observation:

The app separates the civilian and responder workflows at the start. This improves accessibility because users only see the controls relevant to their role. Civilian users are guided through a short emergency request flow, while responders are taken to the incident queue and incident detail screens.

The responder screens use direct action buttons such as View, Accept / Dispatch, Request Support, Flag False Report, and Mark as Done. This makes the workflow easier to follow because each screen has a small number of clear actions.

Limitation:

Some navigation still depends on knowing the correct role and following the intended screen order. A future version could add clearer confirmation messages after every major responder action.

Result:

Pass. The navigation is simple enough for the prototype and supports the main user workflows.

Volunteer Fire Brigade

Civilian

Responder

🔄 Refresh

Incident Detail

Incident: INC-20260525162545

Status: New

View

Updated: 4:25 PM

Incident: INC-20260525153318

Status: New

View

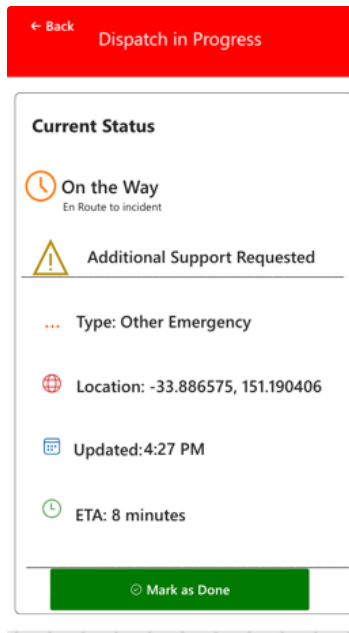
Updated: 3:33 PM

Incident: INC-20260525162555

Status: On the Way

View

Updated: 4:27 PM



AE3 — Input accessibility and error prevention

Observation:

The app prevents some invalid actions before they happen. For example, the civilian request cannot be confirmed until an incident type is selected. This is useful because it avoids creating incomplete incident records in SharePoint. On the responder side, login and action buttons are placed close to the relevant input fields, which makes the flow easier to understand.

Limitation:

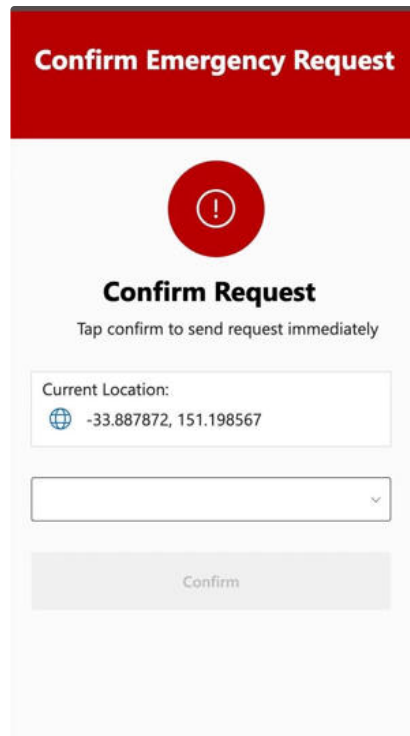
The incident type dropdown is a Power Apps control that Test Studio did not replay reliably. This does not mean the app logic failed, but it shows that dropdown interactions may need manual checking and clearer documentation.

Result:

Pass with limitation noted. Basic input prevention is present, but dropdown behaviour should be manually checked because of Test Studio limitations.

Evidence:

Figure AE3. Confirmation screen showing that the Confirm button remains disabled until a valid incident type is selected.



AE4 — Location information and platform limitation

Observation:

The app captures and displays the user's current GPS coordinates. This gives responders a location reference and keeps the request flow working even when a readable street address cannot be generated.

Limitation:

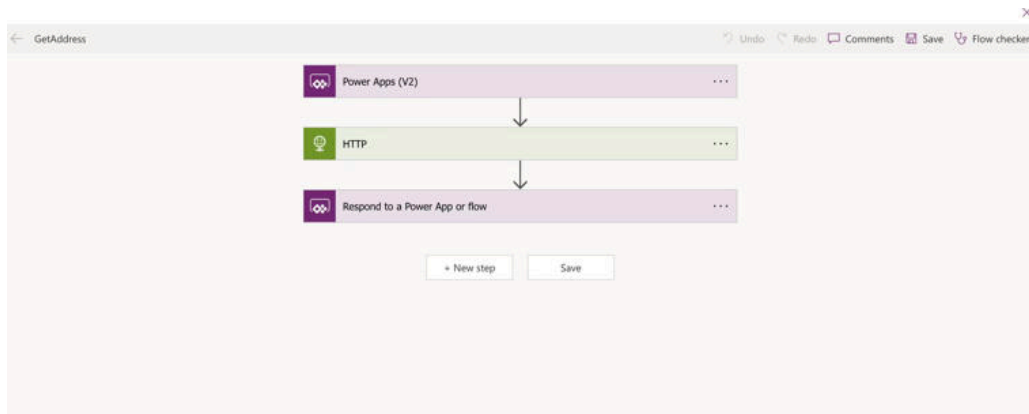
We originally wanted to display the user's location as a readable address instead of raw latitude and longitude. I investigated Azure Maps and also asked on Ed Discussion. The coordinator explained that this process is called reverse geocoding, but Azure Maps is not available in our Power Apps environment due to licensing restrictions. A third-party API such as OpenCage was suggested, but our environment still did not allow us to complete that integration within the assignment scope.

Result:

Pass with documented platform limitation. The app still captures location data, but the prototype displays coordinates rather than a readable address because of connector and permission limitations.

Evidence:

Figure AE4. Location display in the app, showing GPS coordinates as the available fallback.



Issues Found and Fixes

IF means “Issue and Fix”. This section records the main issues found during testing and how we handled them in the final prototype or documentation.

Issue ID	Issue found	Impact on testing / app	Fix or workaround	Final status
IF1	Power Apps Test Studio did not reliably replay the modern incident type dropdown.	In TC1, Test Studio could not navigate to the confirmation screen and show the GPS location, but the dropdown value was not always selected during playback. Because of this, the Confirm button stayed disabled and the final status-screen assertion could fail even though the app	We kept the stable automated steps in Test Studio, including role selection, navigation, and location display. The dropdown selection and final Confirm action were checked manually and documented with screenshots.	Resolved with documented workaround. This was treated as a Test Studio limitation, not a failure of the app logic.

		worked manually.		
IF2	Dynamic incident gallery records were difficult to select consistently in automated tests.	Responder-side tests depend on live SharePoint data. When new records were added or refreshed, Test Studio could sometimes struggle to select the exact same incident card or View button.	We prepared controlled test data in SharePoint before running responder tests. For key flows, we selected incidents by status, such as New or On the Way, and verified the final result using both Test Studio and SharePoint backend evidence.	Resolved with controlled test data and backend verification.
IF3	The app could only display GPS coordinates instead of a readable street address.	This reduced user-friendliness because civilians and responders see latitude and longitude rather than a normal address. However, the core location-	We investigated Azure Maps and asked on Ed Discussion. The coordinator explained that reverse geocoding was needed,	Documented platform limitation. Location capture works, but readable address conversion is listed

		capture function still worked.	but Azure Maps was not available in the university Power Apps environment due to licensing restrictions. A third-party API such as OpenCage was suggested, but our environment still did not allow the integration within the assignment scope. We kept GPS coordinates as the fallback.	as a future improvement.
IF4	Power Apps Monitor showed a CountRows delegation warning.	The warning did not stop the prototype from working, but it suggests that the formula may not scale well if the SharePoint list becomes much larger.	We documented the warning as a scalability issue. For this prototype, the dataset was small	Accepted for prototype , document ed for future optimisati on.

			enough that the tested workflows were not affected. A future version should reduce unnecessary CountRows use and apply more specific filters when loading incident records.	
IF5	Some Test Studio evidence needed manual support because several behaviours were visual or data-dependent.	Behaviours such as password visibility, disabled Confirm button, dropdown options, and GPS fallback were not always meaningful to automate fully in Test Studio.	We separated these into Additional Manual Checks instead of forcing them into automated Test Studio steps. This made the test suite cleaner and avoided misleading failed	Resolved through separate manual check evidence.

			automated steps.	
--	--	--	------------------	--

Overall, the issues found did not block the main prototype workflows. Most issues were caused by Power Apps testing-tool limitations, dynamic SharePoint data, or university environment restrictions. Where the issue affected automated testing, we documented the limitation and used manual screenshots or backend evidence as a workaround. Where the issue affected future scalability or usability, we recorded it as a future improvement rather than hiding it.

Testing Conclusion

Overall, the testing showed that the main emergency response workflows worked as intended in the final prototype. The Power Apps Test Studio suite covered the core civilian and responder flows, including emergency request submission, request validation, responder sign-up and login, incident queue filtering, accepting a dispatch, requesting additional support, flagging false reports, and marking incidents as done.

The backend evidence confirmed that important data changes were saved in SharePoint, including new incident creation, responder account creation, status updates, support request flags, false report status, and completed incident status. This was important because the app needed to demonstrate real integration, not just screen navigation.

Power Apps Monitor was used to evaluate performance behaviour. The main network calls completed successfully, including createRow, getRows, and patchRow operations. The clearest performance issue was the slower first getRows call on the responder overview screen, which took 3334 ms. This was acceptable for the prototype, but it suggests that future versions should optimise the incident gallery and reduce unnecessary list operations.

The main limitations found were related to Power Apps and the university environment rather than the intended app logic. Test Studio did not reliably replay some modern controls, and readable address conversion was not available because Azure Maps and third-party geocoding access were restricted. These limitations were documented with manual checks and workarounds.

Based on the automated tests, manual checks, backend evidence, Monitor observations, and accessibility review, the prototype is ready for assignment submission. The remaining improvements are mainly future enhancements, such as better address conversion, stronger scalability for larger SharePoint lists, and more polished accessibility support for field use.